

AURIX

Microcontroller Infineon Software Architecture

User's Manual

WDG Driver

Release v4.5

2016-05

Edition 2016-05

**Published by
Infineon Technologies AG
81726 Munich, Germany**

**© 2016 Infineon Technologies AG
All Rights Reserved.**

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Date	Version	Change Description
2016-05-19	4.5	Updated for JIRA Id:0000051261-231 (I/O mode updated for USER Mode support for Section 6.2), Section 4.2, Section 4.9, File Structure at Section 3, Interface Diagram at Section 2.2
2015-11-03	4.4	Section 8: Added DEM, WDG_E_INIT_FAILED, updated Section 6.2.1 and 6.2.6. Updated Section 2.2, wrt Interface diagram. Updated Section 4.2.11, wrt SMU state for WDG
2015-09-28	4.3	Updated Code listing 3, wrt password calculation (JIRA 0000051047-189)
2015-06-10	4.2	Update in Sec 6.3.1 to support pulse notify interrupt mode (JIRA 0000051261-77)
2015-05-27	4.1	Updated for REV_007809 in Sec 4.7
2015-05-27	4.0	Updated Note in Sec 4.7 for ATOM channel.(JIRA 0000051047-98)
2015-04-29	3.9	Updated for review comments REV_007617
2015-04-29	3.8	Updated for JIRA 0000051124-49 to make some sections of the user information more clear.
2015-03-09	3.7	Updated based upon Review ID: REV_007174 in sections "4.3.1, 4.3.2, 4.3.3, 6.2.2 and 6.2.3"
2015-03-09	3.6	Updated sections 4.3.1, 4.3.2, 4.3.3, 6.2.2 and 6.2.3 for APIs Wdg_17_Scu_SetMode and Wdg_17_Scu_SetTriggerCondition visibility information when Safety is enabled.
2015-02-10	3.5	Updated based on JIRA: 0000051047-26 Updated sections 4.7.4, 4.7.5, 4.7.6, 4.7.7, 4.7.8 and 4.7.9 for providing information about calculation of WdgSlowServiceGtmCbkTime and WdgFastServiceGtmCbkTime (where x = 0,1 and 2)
2015-01-16	3.4	Updated as per Rev ID: REV_006848. Removed TC23x ITS from 1.4, updated Figure 4 for Wdg_Local.h file, and all APIs visibility based upon Watchdog Timer is configured for QM or Safety mode.
2015-01-16	3.3	Changed API Wdg_17_Scu_SafeTrigger (removed ResetPassword parameter) in section 6.2.4, New API introduced Wdg_17_Scu_InitAltResetPassword in section 6.2.6, added new DET error WDG_E_PARAM_PASSWD in section 8.0, updated sections 10.1, 10.2, 9.2.1 and 9.4.2 based on UTP: AI00253147 Updated parameter WdgUnlockRestriction parameter in section 4.2.11 based on UTP: AI00253342. Updated Figure 2 and Figure 3 in Section 2.2 and limitations are updated in Section 11.2
2014-11-10	3.2	Updated for UTP: AI00251090
2014-09-22	3.1	Updated for UTP AI00249496
2014-08-13	3.0	Updated as per review comments REV_006007
2014-08-13	2.9	Updated for UTP AI00247533
2014-07-14	2.8	Modified first paragraph of Safety Information Section 10

CONFIDENTIAL

2014-05-06	2.7	SFR Modified option is made to NONE for WdgInitialTimeout and WdgMaxTimeout.
2014-02-26	2.6	Modified the References
2014-02-28	2.5	Updated for an EP device changes
2013-11-05	2.4	Updated with imetrix review comments REV_004854
2013-08-26	2.3	Safety changes updated
2013-07-12	2.2	WDG_MAX_TIMERS added in derived configuration parameters section
2013-04-01	2.1	Updated for CommonPublishedInformation
2013-03-13	2.0	Updated for Multicore and B step changes
2012-12-18	1.5	Updated file structure
2012-10-30	1.4	Modified Description of WdgSettingsConfig
2012-08-13	1.3	Removed all AS321 related descriptions
2012-06-15	1.2	Updated example usage. Added sec 8.2 on WDG operation.
2012-05-14	1.1	Replaced GPT to GTM settings
2011-11-09	1.0	File version baselined to 1.0
2011-11-09	0.4	Vendor ID changes done.
2011-10-21	0.3	STC timer concept is replaced by timeout counter concept for the detection of triggering timeout which is nothing but the bigger window.
2011-10-04	0.2	Configuration parameters are updated
2011-08-04	0.1	Initial version

We Listen to Your Comments

Any information within this document that you feel is wrong, unclear or missing at all?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

<mailto:mcdocu.comments@infineon.com>



Table of Contents		Page
1	Introduction	12
1.1	Scope	12
1.2	Abbreviations.....	12
1.3	Definitions.....	13
1.4	References	13
2	Driver Overview	14
2.1	Software Hardware Mapping.....	14
2.1.1	SYSTEM CLOCK	14
2.1.2	GTM	15
2.1.3	WDT	15
2.2	Software Driver Description	15
3	File Structure	18
4	Configuration Documentation	21
4.1	Configuration Concept	21
4.1.1	Configuration Class	21
4.1.2	Configuration Variant	21
4.1.3	How to read the Configuration Class field.....	21
4.2	Container: WdgGeneral	22
4.2.1	WdgDevErrorDetect	22
4.2.2	WdgDisableAllowed	22
4.2.3	WdgTriggerLocation	23
4.2.4	WdgVersionInfoApi	23
4.2.5	WdgIndex	24
4.2.6	WdgPBFixedAddress	24
4.2.7	WdgInitialTimeout	24
4.2.8	WdgMaxTimeout	25
4.2.9	Debugging support	25
4.2.10	WdgRunArea.....	26
4.2.11	WdgUnlockRestriction	26
4.2.12	WdgRunningInUser0Mode.....	27
4.2.13	WdgUserModeInitApiEnable	27
4.2.14	WdgUserModeRuntimeApiEnable	28
4.3	Container: WdgSafety	28
4.3.1	Wdg0SafetyEnable	28
4.3.2	Wdg1SafetyEnable	29
4.3.3	Wdg2SafetyEnable	29
4.3.4	WdgSetOffModeApi	30
4.3.5	WdgEnableTimecheck	30
4.4	Container: WdgDemEventParameterRefs	31
4.4.1	WdgDisableRejectedDemRef	31
4.4.2	WdgModeSwitchFailedDemRef	31
4.5	Container: WdgSettingsConfig.....	32
4.5.1	WdgSystemClockRef	32
4.5.2	WdgDefaultMode	32
4.5.3	Wdg1DefaultMode	33
4.5.4	Wdg2DefaultMode	33
4.5.5	Wdg0FastModeTimeoutValue.....	34
4.5.6	Wdg1FastModeTimeoutValue.....	34
4.5.7	Wdg2FastModeTimeoutValue.....	35
4.5.8	Wdg0SlowModeTimeoutValue	35
4.5.9	Wdg1SlowModeTimeoutValue	36
4.5.10	Wdg2SlowModeTimeoutValue	36
4.5.11	Container: WdgInitialPassword	36
4.5.11.1	Wdg0InitialPassword.....	36
4.5.11.2	Wdg1InitialPassword.....	37
4.5.11.3	Wdg2InitialPassword.....	37

Table of Contents	Page
4.5.12 Container: WdgTimerCheckTolerance	38
4.5.12.1 Wdg0TimerTolerance	38
4.5.12.2 Wdg1TimerTolerance	38
4.5.12.3 Wdg2TimerTolerance	38
4.6 Container: WdgExternalConfiguration	39
4.6.1 WdgExternalContainerRef	39
4.7 Container: WdgGtmSettings	39
4.7.1 Wdg0ServiceGtmChannelRef	39
4.7.2 Wdg1ServiceGtmChannelRef	40
4.7.3 Wdg2ServiceGtmChannelRef	40
4.7.4 Wdg0SlowServiceGtmCbkJTime	41
4.7.5 Wdg1SlowServiceGtmCbkJTime	42
4.7.6 Wdg2SlowServiceGtmCbkJTime	42
4.7.7 Wdg0FastServiceGtmCbkJTime	43
4.7.8 Wdg1FastServiceGtmCbkJTime	44
4.7.9 Wdg2FastServiceGtmCbkJTime	45
4.8 Container: CommonPublishedInformation	46
4.8.1 ArMajorVersion	46
4.8.2 ArMinorVersion	46
4.8.3 ArPatchVersion	46
4.8.4 SwMajorVersion	47
4.8.5 SwMinorVersion	47
4.8.6 SwPatchVersion	48
4.8.7 VendorId	48
4.8.8 ModuleId	48
4.8.9 Release	49
4.9 Derived Configuration Parameters	49
4.9.1 WDG_MAXSAFE_TIMERS	49
4.9.2 WDG_MAX_TIMERS	49
4.9.3 WDG_MAXQM_TIMERS	50
4.9.4 WDG_RUNNING_IN_USER_0_MODE_ENABLE	50
4.9.5 WDG_USER_MODE_INIT_API_ENABLE	51
4.9.6 WDG_USER_MODE_RUNTIME_API_ENABLE	51
4.10 Property File Variables	52
4.10.1 Wdg.MaxTimers	52
5 Published parameters	53
5.1 WDG Trigger Mode	53
5.2 VendorId	53
5.3 ModuleId	53
5.4 SwMajorVersion	53
5.5 SwMinorVersion	54
5.6 SwPatchVersion	54
5.7 ARMajorVersion	54
5.8 ARMinorVersion	54
5.9 ARPatchVersion	54
6 API Documentation	56
6.1 API Type Definitions	56
6.1.1 Type Definition Wdg_ConfigType	56
6.1.2 Type Definition Wdg_StatusType	56
6.2 API Functions	56
6.2.1 Wdg_17_Scu_Init	56
6.2.2 Wdg_17_Scu_SetMode	58
6.2.3 Wdg_17_Scu_SetTriggerCondition	59
6.2.4 Wdg_17_Scu_SafeTrigger	60
6.2.5 Wdg_17_Scu_SetOffMode	61
6.2.6 Wdg_17_Scu_InitAltResetPassword	62
6.2.7 Wdg_17_Scu_GetVersionInfo	64

Table of Contents	Page
6.3 Callback Functions	64
6.3.1 Wdg_17_Scu_Cbk_GtmNotifySRV	64
6.4 Interrupt Handling	65
6.5 Exclusive Area (EA) Information	66
6.5.1 Wdg_Trigger	66
6.5.2 Wdg_TimerHandling	66
6.5.3 Wdg_ChangeMode	66
7 Data Description	67
8 Error classification	68
9 Example Usage	69
9.1 Configuration of Driver	69
9.2 Watchdog Operation	69
9.2.1 Watchdog Operation	69
9.3 Initialization of Driver	70
9.4 Using the API	73
9.4.1 Changing Mode of Operation	73
9.4.2 Triggering Watchdog	73
9.4.3 Disabling Watchdog	74
9.4.4 Set Watchdog to OFF mode	74
10 Safety Information	75
10.1 Safety Mechanisms	75
10.2 Condition of use	75
10.3 Sequence diagrams	77
11 Assumptions and Limitations	79
11.1 Assumptions and Deviations from the Software Specification	79
11.2 Limitations	79
11.3 Hardware Features Not Supported	79

List of Figures**Page**

Figure 1	SW HW Driver Mapping	14
Figure 2	Interface diagram of WDG Driver	16
Figure 3	Interface diagram of WDG Driver for AS4.0.3	17
Figure 4	File Structure of WDG Driver(Safety Disabled)	18
Figure 5	Code Structure of WDG Driver (Safety Enabled)	19
Figure 6	WDG initialization	77
Figure 7	Access to Mcal_ResetENDINIT()/Mcal_SetENDINIT() after WDG initialization	78

List of Tables

Page

Table 1	Abbreviations Used in this Document	12
Table 2	Definitions.....	13
Table 3	File Contents	19
Table 4	Configuration Class Example 1.....	22
Table 5	Configuration Class Example 2.....	22
Table 6	WdgDevErrorDetect	22
Table 7	WdgDisableAllowed	22
Table 8	WdgTriggerLocation	23
Table 9	WdgVersionInfoApi	23
Table 10	WdgIndex	24
Table 11	Wdg Post build fixed address feature	24
Table 12	WdgInitialTimeout	24
Table 13	WdgMaxTimeout	25
Table 14	WdgDebugSupport.....	25
Table 15	WdgRunArea.....	26
Table 16	WdgUnlockRestriction	26
Table 17	WdgRunningInUser0Mode.....	27
Table 18	WdgUserModeInitApiEnable	27
Table 19	WdgUserModeRuntimeApiEnable	28
Table 20	Wdg0SafetyEnable	28
Table 21	Wdg1SafetyEnable	29
Table 22	Wdg2SafetyEnable	29
Table 23	WdgSetOffModeApi	30
Table 24	WdgEnableTimecheck	30
Table 25	WDG_E_DISABLE_REJECTED.....	31
Table 26	WDG_E_MODE_FAILED.....	31
Table 27	WdgSystemClockRef	32
Table 28	WdgDefaultMode	32
Table 29	Wdg1DefaultMode	33
Table 30	Wdg2DefaultMode	33
Table 31	Wdg0FastModeTimeoutValue.....	34
Table 32	Wdg1FastModeTimeoutValue.....	34
Table 33	Wdg2FastModeTimeoutValue.....	35
Table 34	Wdg0SlowModeTimeoutValue	35
Table 35	Wdg1SlowModeTimeoutValue	36
Table 36	Wdg2SlowModeTimeoutValue	36
Table 37	Wdg0InitialPassword.....	36
Table 38	Wdg1InitialPassword.....	37
Table 39	Wdg2InitialPassword.....	37
Table 40	Wdg0TimerTolerance.....	38
Table 41	Wdg1TimerTolerance.....	38
Table 42	Wdg2TimerTolerance.....	38
Table 43	WdgExternalContainerRef	39
Table 44	Gtm channel reference to service watchdog hardware (WDT0).....	39
Table 45	Gtm channel reference to service watchdog hardware (WDT1).....	40
Table 46	Gtm channel reference to service watchdog hardware (WDT2).....	40
Table 47	Slow Service callback period of Gtm for WDT0	41
Table 48	Slow Service callback period of Gtm for WDT1	42
Table 49	Slow Service callback period of Gtm for WDT2	42
Table 50	Fast Service callback period of Gtm for WDT0.....	43
Table 51	Fast Service callback period of Gtm for WDT1.....	44
Table 52	Fast Service callback period of Gtm for WDT2	45
Table 53	ArMajorVersion.....	46
Table 54	ArMinorVersion.....	46
Table 55	ArPatchVersion	46
Table 56	SwMajorVersion	47
Table 57	SwMinorVersion	47
Table 58	SwPatchVersion	48

List of Tables

Page

Table 59	VendorId.....	48
Table 60	ModuleId.....	48
Table 61	Release	49
Table 62	WDG_MAXSAFE_TIMERS.....	49
Table 63	WDG_MAX_TIMERS	49
Table 64	WDG_MAXQM_TIMERS	50
Table 65	WDG_RUNNING_IN_USER_0_MODE_ENABLE.....	50
Table 66	WDG_USER_MODE_INIT_API_ENABLE.....	51
Table 67	WDG_USER_MODE_RUNTIME_API_ENABLE	51
Table 68	Wdg.MaxTimers	52
Table 69	WDG Trigger Mode	53
Table 70	VendorId.....	53
Table 71	ModuleId.....	53
Table 72	SwMajorVersion	53
Table 73	SwMinorVersion	54
Table 74	SwPatchVersion	54
Table 75	ARMajorVersion	54
Table 76	ARMinorVersion	54
Table 77	ARPatchVersion	54
Table 78	Type Definition Wdg_ConfigType	56
Table 79	Type Definition Wdg_StatusType.....	56
Table 80	Specification of API Wdg_17_Scu_Init	56
Table 81	Specification of API Wdg_17_Scu_SetMode.....	58
Table 82	Specification of API Wdg_17_Scu_SetTriggerCondition	59
Table 83	Specification of API Wdg_17_Scu_SafeTrigger	60
Table 84	Specification of API Wdg_17_Scu_SetOffMode	61
Table 85	Specification of API Wdg_17_Scu_InitAltResetPassword.....	62
Table 86	Specification of API Wdg_17_Scu_GetVersionInfo	64
Table 87	Specification of callback Wdg_17_Scu_Cbk_GtmNotifySRV	64
Table 88	Specification of Wdg_Trigger	66
Table 89	Specification of Wdg_TimerHandling	66
Table 90	Specification of Wdg_ChangeMode.....	66
Table 91	Global Data Description	67
Table 92	Error Classification	68

List of Code Listings

Page

Code Listing 1	WDG Driver Initialization	70
Code Listing 2	Example Configuration	70
Code Listing 3	Password calculation.....	73
Code Listing 4	Change mode operation	73
Code Listing 5	Triggering WDG.....	73
Code Listing 6	Triggering WDG with safety	74
Code Listing 7	Disable Watchdog	74
Code Listing 8	SetOffMode Watchdog	74

1 Introduction

This User Manual provides information regarding the functionality, configuration parameters and API implementation for the WDG Driver.

The WDG Driver is built according to the AUTOSAR specification.[2](#)

This User Manual is intended to help the user to get acquainted with the WDG Driver implementation for the AURIX Family of microcontrollers. It is the documentation that describes how to use the WDG Driver.

1.1 Scope

This document addresses the following features of the WDG Driver implementation,

- AURIX microcontroller HW peripherals used to realize the Driver
- File structure of WDG Driver
- Configuration Parameters of the Driver
- Parameters published by WDG Driver
- Resource usage
- API Specification
- Code Snippets illustrating the common use cases.
- Limitations and Assumptions

This documentation applies to the WDG Driver Variant: Variant PB

1.2 Abbreviations

The abbreviations used inside this document are explained in [Table 1](#).

Table 1 Abbreviations Used in this Document

Abbreviation	Explanation
μC	Microcontroller
μs	Microsecond
API	Application Programming Interface
AUTOSAR	AUTomotive Open System Architecture
MC-ISAR	Microcontroller Infineon Software Architecture
BSW	Basic Software
DET	Development Error Tracer
DS	Design Specification
EP	Effeciency Platform
HW	Hardware
IFX	Infineon Technologies
MCAL	MicroController Abstraction Layer
MCU	MicroController Unit
SCU	System Control Unit
SMU	Safety Management Unit
SW	Software
TTL	Transistor Transistor Logic
WDGx	Watchdog timer for CPUx (x = 0,1,2 for TC27x,TC29x and 0,1 for TC26x, 0 for TC23x)
WDTx	Watchdog Timer

STC	Short form of the API Wdg_17_Scu_SetTriggerCondition. This also represents the window period that opens after calling this API. This is typically carried out by a dedicated Gtm channel.
SRV	Stands for Service of the hardware WDG. This is typically carried out by a dedicated Gtm channel.
GTM	Generic Timer Module
TOM	Timer Output Module
ATOM	ARU-connected Timer Output Module

1.3 Definitions

The following definitions are provided:

Table 2 Definitions

Keyword	Definition
Password	Also termed as Signature, that follows a pseudo-random sequence based upon a 14-bit Fibonacci LFSR (Linear Feedback Shift Register) with characteristic polynomial $x^{14}+x^{13}+x^{12}+x^2+1$.

1.4 References

This section lists down all relevant documents for this User Manual. Some of the artifacts listed below are given without the version number. For those artifacts, kindly refer the latest version.

1. Requirements on WDG :
AUTOSAR_SRS_WDG_Driver.pdf
2. Specification of WDG
AUTOSAR_SWS_WatchdogDriver.pdf (Rel 4.0.3).
3. Aurix_MC-ISAR_UM_MCUDriver.doc.
4. Target TC27x User Manual: tc27xD_um_v2.2.pdf
5. User Manual: tc23x_tc22x_um_v1.1.pdf
6. Aurix_MC-ISAR_UM_GTMDriver.pdf
7. Target TC26x User Manual: tc26xB_um_v1.3.pdf
8. Target TC29x User Manual: tc29xB_um_v1.2_2.pdf

2 Driver Overview

The WDG Driver provides the services and configuration parameters required to use the CPU WDT peripheral of AURIX family of microcontrollers. It provides functions to service the CPU WDT and suitably change the configuration of CPU WDT.

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) illustrates the Hardware peripherals and their interaction with the Software Drivers.

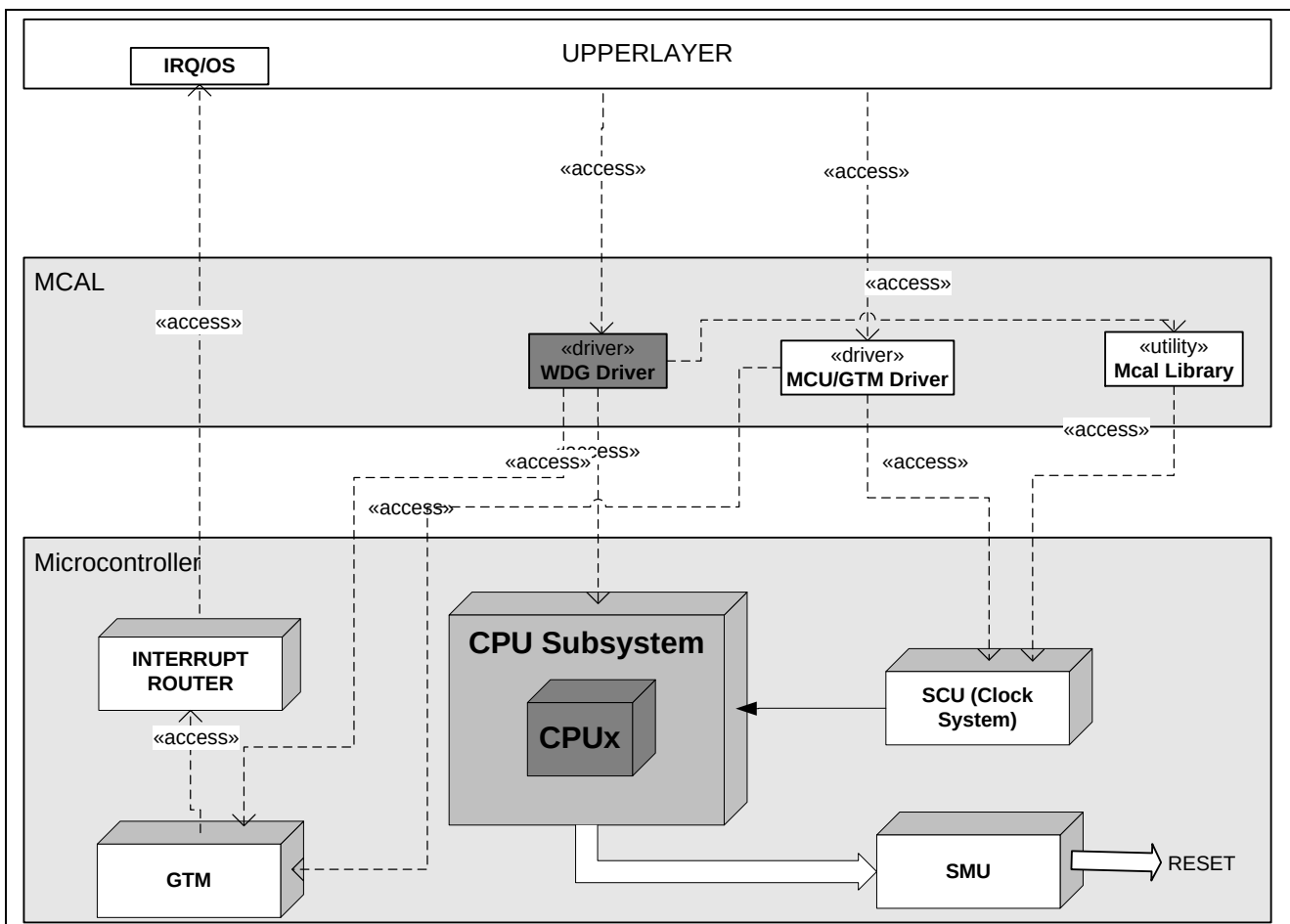


Figure 1 SW HW Driver Mapping

2.1.1 SYSTEM CLOCK

The System Clock is set up via the MCU driver. This is mandatory for the user to set up an appropriate system clock. Note that the WDT reload value calculation for Fast mode and Slow mode depends on the System Clock configured.

Refer [3](#) for details about MCU Configuration.

Refer [6](#) for details about GTM Configuration.

2.1.2 GTM

There is one Gtm channel per CPU WDG, that is necessary for the functioning of Wdg module. For each CPU WDG one GTM channel has to be configured. This channel takes care of the SRV time period for respective WDG.

SRV time period behaves like heart of the complete timing control of the Wdg functionality.

2.1.3 WDT

The WDT peripheral contains a 16bit up-counter which can operate at two prescaled operating frequencies $f_{\text{SYS}}/256$ (WDGIF_FAST_MODE) and $f_{\text{SYS}}/16384$ (WDGIF_SLOW_MODE), where f_{SYS} is system clock to the WDT peripheral. In the **Normal** mode, the WDT starts counting up from a user-defined reload value. The WDT needs to be serviced by the software before the counter reaches 0xFFFF to prevent the WDT from triggering the SMU. SMU will respond based on its configuration (if reset is configured it will reset the system)

The WDT also access of critical registers using the ENDINIT bit of the WDTCPUXCON0 register.

A two step procedure needs to be executed every time for modifying the WDTCPUXCON0 register. The procedure comprises

1. **Password Access:** The password is written to the WDTCPUXCON0 register. If it matches the expected password, it is unlocked for modification.
2. **Modify Access:** After a successful password access, the WDTCPUXCON0 register can be modified using the Modify Access. The desired ENDINIT bit, user defined password and reload value along with fixed guard bits are used to determine the value to be written during Modify Access. After this access is completed, the Watchdog lock is SET and hence any modification to the WDTCPUXCON0 register requires an execution of the Password Access.

Apart from the ENDINIT bit, the WDT provides another safety feature by limiting the duration to access the critical registers. As soon as the Password Access is successful, WDT enters the **Timeout** mode and the WDT starts counting up from 0xFFFC. In this mode, the ENDINIT bit can be reset (0) via the Modify Access for accessing the critical registers. However, the ENDINIT bit needs to be set (1) before WDT overflows. The overflow of WDT triggers the SMU. SMU will respond based on its configuration (if reset is configured it will reset the system).

Note: TC23x provides 2 Wdg modules Viz WDT0 and WDTs. This driver only supports WDT0.

ATOM is not available in TC23x. WDG driver for TC23x will not support ATOM.

2.2 Software Driver Description

The [Figure 2](#) and [Figure 3](#) shows the Application Programming Interface (API) for the WDG Driver and its interaction with other Drivers.

Development Error Tracer (DET) is required if DET is enabled for the WDG Driver.

The Configuration is further explained in Chapter 4. The API's are explained in Chapter 5.9.

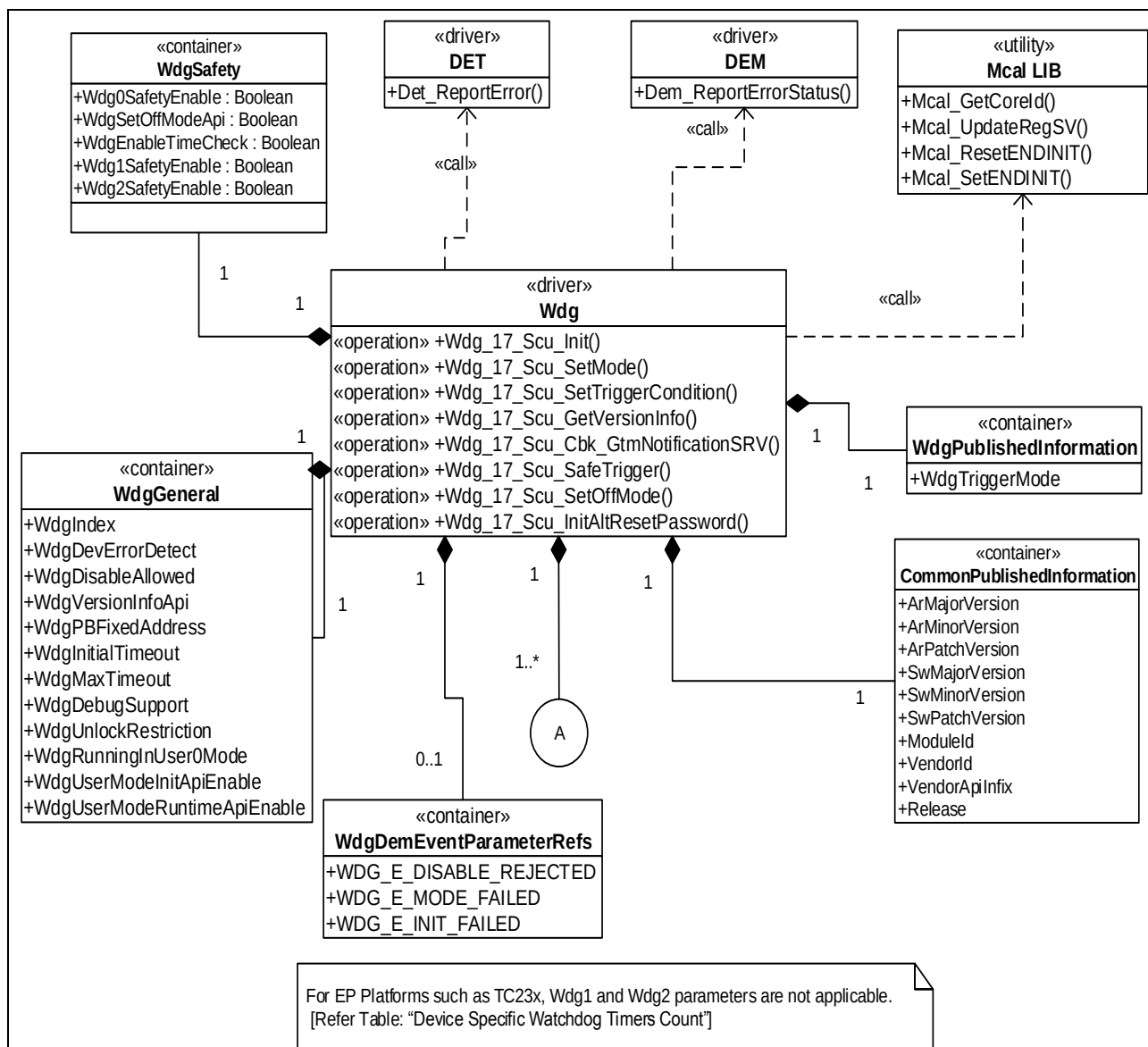


Figure 2 Interface diagram of WDG Driver

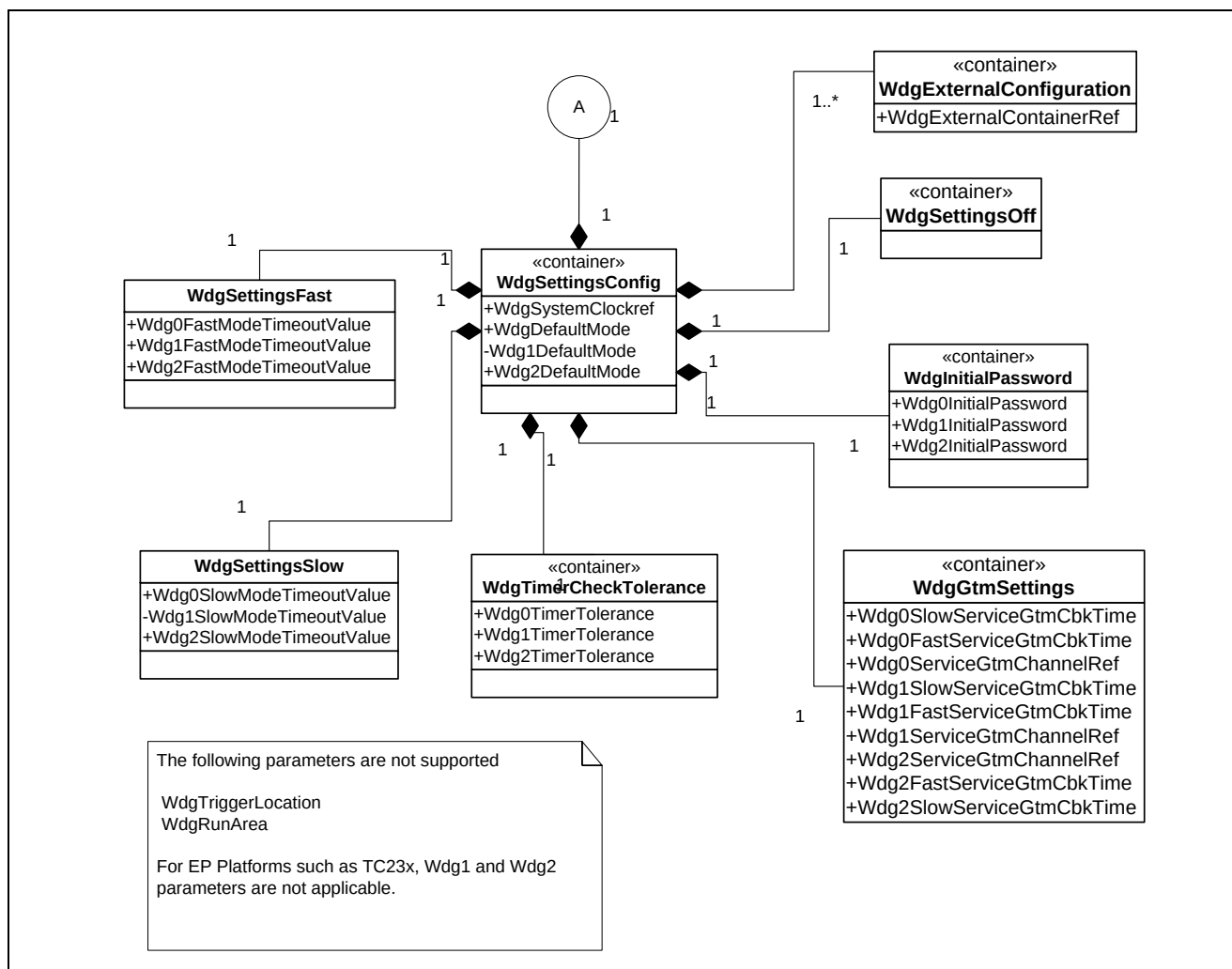


Figure 3 Interface diagram of WDG Driver for AS4.0.3

3 File Structure

This section provides details about the WDG Driver files and other related files.

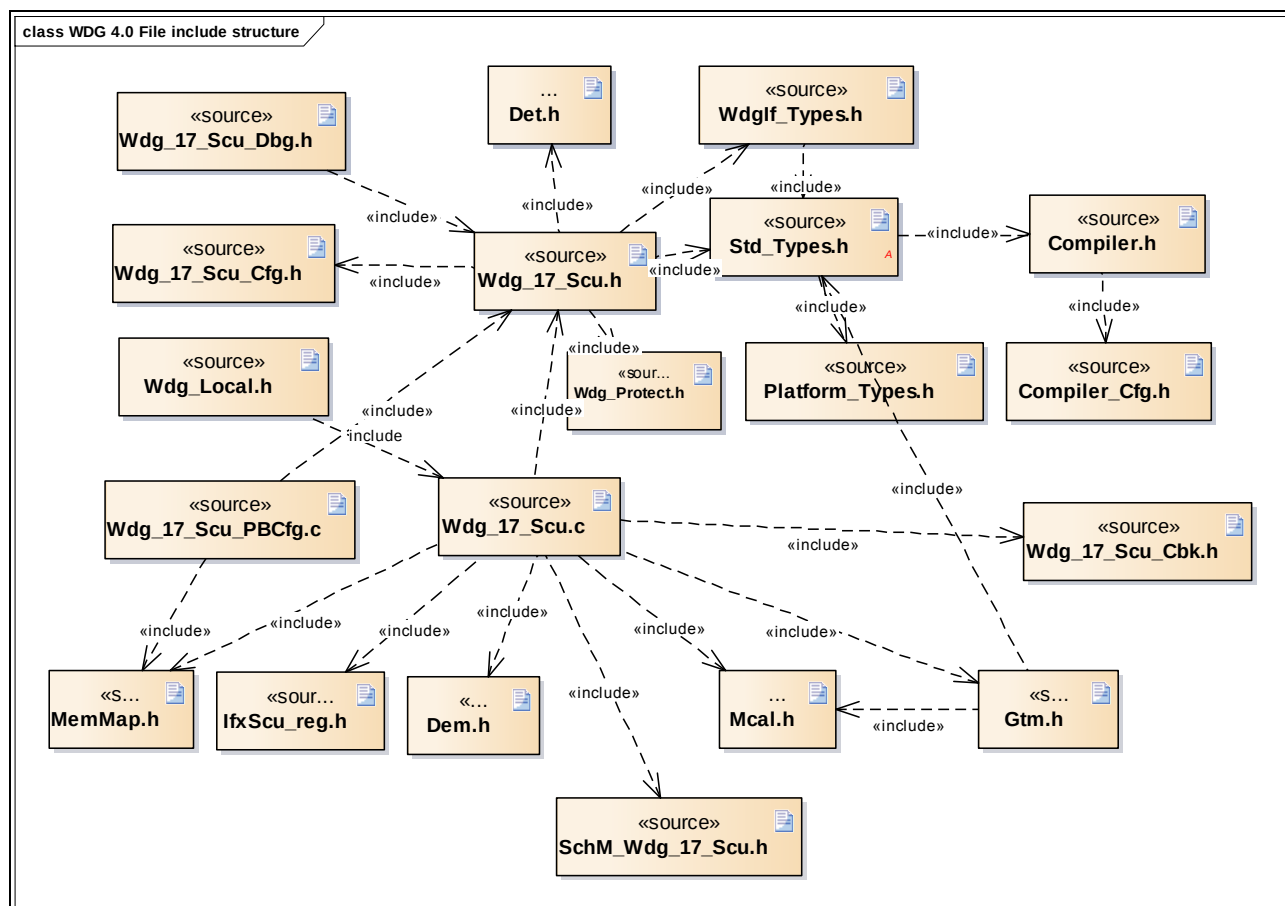


Figure 4 File Structure of WDG Driver(Safety Disabled)

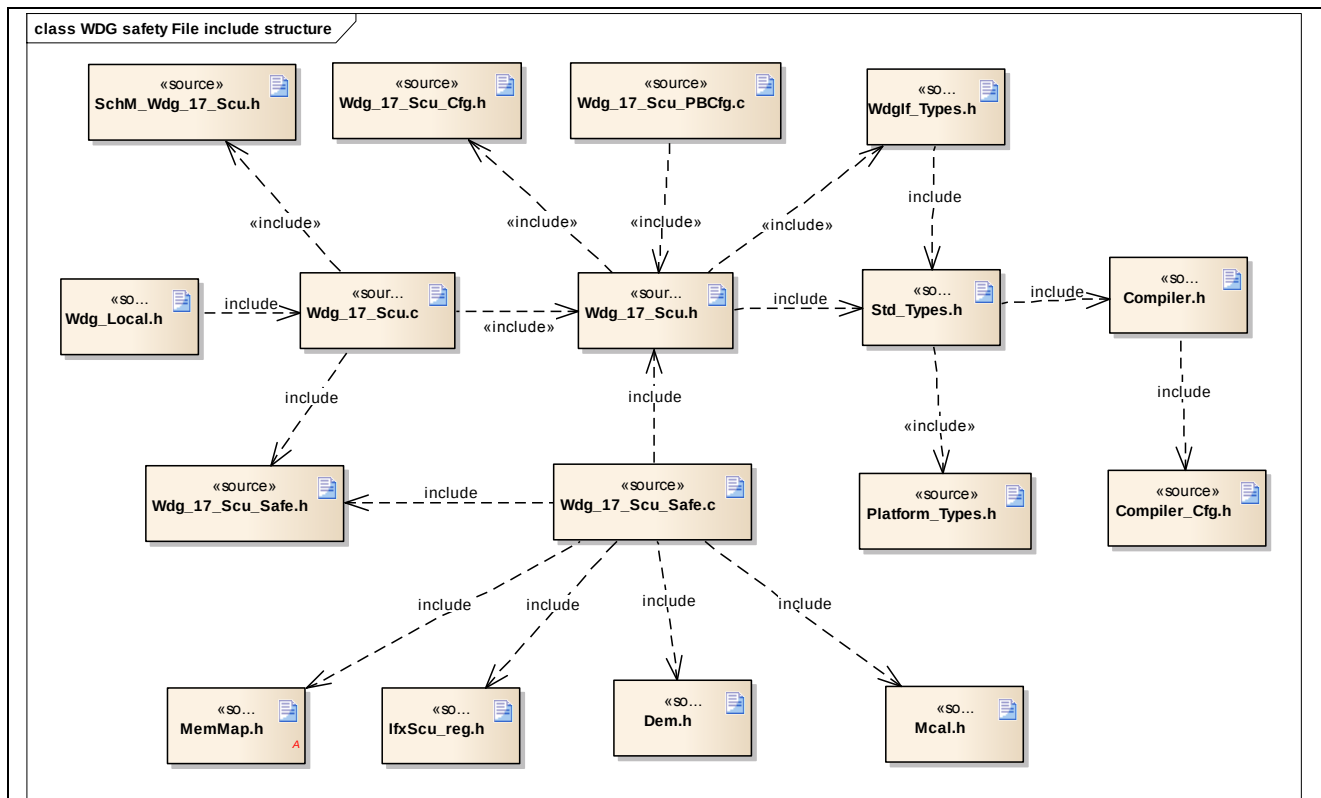


Figure 5 Code Structure of WDG Driver (Safety Enabled)

Table 3 File Contents

File Name	File Contents
Platform_Types.h	Platform Dependencies are declared here.
Compiler.h	Tasking specific Compiler abstraction is declared here.
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code within declaration and definition.
Std_Types.h	Standard Data types to be used are declared here.
lfxScu_reg.h	Includes the register definition file for microcontroller.
Mcal.h	Provides library functions for MCAL layer.
SchM_Wdg.h	Provides the exclusive area functions.
Det.h	Provides the functionality of Development Error tracer.
Dem.h	Provides the functionality of Diagnostic Error Manager.
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections.
Wdg_17_Scu.h	Exports WDG Driver functionality.
Wdg_17_Scu_Local.h	Exports GTM/ATOM timer disable/enable functionality.
Wdg_17_Scu_Safe.h	Exports WDG safety functionality
Wdg_17_Scu.c	Implementation of WDG Driver functionality.
Wdg_17_Scu_Safe.c	Implementation of WDG safety functionality
Wdg_17_Scu_Cfg.h	Provides Pre Compile Configuration data of WDG Driver.
Wdg_17_Scu_PBCfg.c	Provides Post Build configuration data of WDG Driver.
WdgIf_Types.h	Provides the interface data types for WDG Driver.
Gtm.h	Provides the interfaces for all timing based tracking of WDT.

Wdg_17_Scu_Dbg.h	Provides extern declarations for debug related variables.
Wdg_17_Scu_Cbk.h	Provides extern for call back function
Wdg_Protect.h	Included to define all the first level indirection macros for Protected mode support for supervisory access via OS calls

4 Configuration Documentation

The following chapters summarize

- All configuration parameters and their description
- The type definition used to represent them in the code.

4.1 Configuration Concept

4.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

- Compiling,
- Linking and
- Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be frozen before compilation and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

NOTE The WDG Driver is delivered as a Variant PB

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

NOTE Topics **4.1.1** , **4.1.2** should be read for understanding the configuration class field

Table 4 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, Variant PC.

Table 5 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a

3. pre-compile parameter if the delivery type is Variant PC delivery
4. post-build parameter if the delivery type is a Variant PB delivery

4.2 Container: WdgGeneral

This container lists down all the general configuration parameters for WDG Driver.

4.2.1 WdgDevErrorDetect

Table 6 WdgDevErrorDetect

Name:	WdgDevErrorDetect		
Source:	This parameter is defined by the AUTOSAR, refer 2 .		
File:	Wdg_17_Scu_Cfg.h		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class:	Pre-compile	x	Variant PB
	Link Time	--	--
	Post build	--	--
Description:	Switches the Development Error Detection and Notification TRUE or FALSE.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.2 WdgDisableAllowed

Table 7 WdgDisableAllowed

Name:	WdgDisableAllowed
Source:	This parameter is defined by AUTOSAR, refer 2 .
File:	Wdg_17_Scu_Cfg.h

Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	Compile switch to allow/forbid disabling the Watchdog Driver at runtime.		
SFRs Modified	WDTCPUxCON1		
Scope:	Driver		
Dependency:	None		

4.2.3 WdgTriggerLocation

Table 8 WdgTriggerLocation

Name:	WdgTriggerLocation		
Source:	This parameter is defined by the AUTOSAR, refer 2 .		
File:	--		
Range:	--		
Configuration Class	Pre-Compile	x	All variants
	Link Time	--	--
	Post Build	--	--
Description:	This field provides the location (memory address) where WDG trigger routine is present. As this feature is not provided by the AURIX microcontroller, it is not editable.		
SFRs Modified	None		
Scope:	--		
Dependency:	--		

4.2.4 WdgVersionInfoApi

Table 9 WdgVersionInfoApi

Name:	WdgVersionInfoApi		
Source:	This parameter is defined by the AUTOSAR, refer 2 .		
File:	Wdg_17_Scu_Cfg.h		
Range:	Values: TRUE: Version info API enabled FALSE: Version Info API disabled Default Value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable / disable the API to read out the version information of the Driver		
SFRs Modified	None		

Scope:	Driver
Dependency:	None

4.2.5 WdgIndex

Table 10 WdgIndex

Name:	WdgIndex		
Source:	This parameter is defined by the AUTOSAR, refer 2 .		
File:	--		
Range:	--		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Wdg driver's ID used by WdgInterface as a reference.		
SFRs Modified	None		
Scope:	--		
Dependency:	--		

4.2.6 WdgPBFixedAddress

Table 11 Wdg Post build fixed address feature

Name:	WdgPBFixedAddress		
Source:	This parameter is microcontroller specific.		
File:	Wdg_17_Scu_Cfg.h		
Range:	Values: TRUE – Fixed address feature used FALSE – Fixed address feature not used Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	The fixed address feature supports the optimization of a post build configuration with a fixed address by avoiding unnecessary indirection.		
SFRs Modified	None		
Scope:	--		
Dependency:	--		

4.2.7 WdgInitialTimeout

Table 12 WdgInitialTimeout

Name:	WdgInitialTimeout
Source:	This parameter is defined by AUTOSAR.
File:	Wdg_17_Scu_Cfg.h
Range:	Range: Greater than 1ms and less than WdgMaxTimeout

	Default: 5.0secs		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--
Description:	This is the initial STC window period that is active as soon Wdg_17_Scu_Init is called. This is common parameter for three CPU WDG's		
SFRs Modified	None		
Scope:	Driver		
Dependency:	WdgMaxTimeout		

4.2.8 WdgMaxTimeout

Table 13 WdgMaxTimeout

Name:	WdgMaxTimeout		
Source:	This parameter is defined by AUTOSAR.		
File:	Wdg_17_Scu_Cfg.h		
Range:	Range: 0.001s to 65.535s Default: 32.0secs		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--
Description:	This is the maximum span of STC window.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.9 Debugging support

Table 14 WdgDebugSupport

Name:	WdgDebugSupport		
Source:	This parameter is defined by AUTOSAR.		
File:	Wdg_17_Scu_Cfg.h		
Range:	Values: TRUE - Debug support enabled FALSE - Debug support disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for enabling/disabling the debugging support. #define WDG_DEBUG_SUPPORT (STD_OFF/ON) When this parameter is set to TRUE, the following identifiers get exposed to the debugging module. 1. Wdg_TimeoutCounter: This parameter holds initial timeout or STC timeout		

	value in terms of Gtm callback period of current mode. Example: If Initial timeout value is 20ms with slow mode as default mode and WdgSlowServiceGtmCbKTime having the value of 5ms, in that case Wdg_TimeoutCounter will have the value of 4. 2. Wdg_DriverState : This parameter gives current status of Wdg driver. Please refer section 6.1.2, for various driver states. 3. Wdg_DriverMode: This parameter holds the current running mode of Wdg.
SFRs Modified	None
Scope:	Driver
Dependency:	None

4.2.10 WdgRunArea

Table 15 WdgRunArea

Name:	WdgRunArea		
Source:	This parameter is defined by AUTOSAR		
File:	NA		
Range:	Values: RAM ROM Default value: ROM		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Setting to differentiate the memory area from which Wdg code would be executing. This parameter is not applicable as the executable is independent of the memory area.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.2.11 WdgUnlockRestriction

Table 16 WdgUnlockRestriction

Name:	WdgUnlockRestriction		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_Cfg.h		
Range:	TRUE - Watchdog driver is enabled with SMU restriction of WDT unlock FALSE - Watchdog driver is disabled with SMU restriction of WDT unlock Default value: TRUE		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--

Description:	Request to enable/disable SMU restriction of WDT unlock. Note: If WdgUnlockRestriction is enabled, Safety Management Unit (SMU) should be in RUN state before calling WDG APIs.
SFRs Modified	None
Scope:	Driver
Dependency:	None

4.2.12 WdgRunningInUser0Mode

Table 17 WdgRunningInUser0Mode

Name:	WdgRunningInUser0Mode		
Source:	This parameter is microcontroller specific		
File:	Wdg_17_Scu.xdm / Wdg_17_Scu.bmd		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If 'WdgRunningInUser0Mode' is enabled then the API's selected by 'WdgUserModeInitApiEnable' or 'WdgUserModeRuntimeApiEnable' will run in user mode 0. If the 'WdgRunningInUser0Mode' switch is not selected and if any of the switches 'WdgUserModeInitApiEnable', 'WdgUserModeRuntimeApiEnable' are selected then those APIs will run in user mode 1. This switch selects between user mode0 and user mode 1 WdgRunningInUser0Mode – STD_ON implies User Mode 0 WdgRunningInUser0Mode – STD_OFF implies User Mode 1 if any of the switches (WdgUserModeInitApiEnable or WdgUserModeRuntimeApiEnable) are true else Supervisor Mode (If all the four switches mentioned are OFF).		
Scope:	Driver		
Dependency:	If 'WdgRunningInUser0Mode' is enabled then at least any one of the switches 'WdgUserModeInitApiEnable' or 'WdgUserModeRuntimeApiEnable' should be enabled.		

4.2.13 WdgUserModeInitApiEnable

Table 18 WdgUserModeInitApiEnable

Name:	WdgUserModeInitApiEnable
Source:	This parameter is microcontroller specific
File:	Wdg_17_Scu.xdm / Wdg_17_Scu.bmd

Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If WdgUserModeInitApiEnable is enabled then the WDG Init API will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch WdgRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.2.14 WdgUserModeRuntimeApiEnable

Table 19 WdgUserModeRuntimeApiEnable

Name:	WdgUserModeRuntimeApiEnable		
Source:	This parameter is microcontroller specific		
File:	Wdg_17_Scu.xdm / Wdg_17_Scu.bmd		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If WdgUserModeRuntimeApiEnable is enabled then all APIs other than WDG Init API will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch WdgRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.3 Container: WdgSafety

This container holds safety configuration parameters of WDG driver. Multiplicity 1:1

4.3.1 Wdg0SafetyEnable

Table 20 Wdg0SafetyEnable

Name:	Wdg0SafetyEnable
Source:	This parameter is defined by IFX
File:	Wdg_17_Scu_Cfg.h
Range:	Values: STD_ON STD_OFF

	Default value: STD_OFF		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--
Description:	Used to enable safety features of WDG0. NOTE: 1. If this parameter is enabled then user shall ensure that TOM/ATOM channels are not configured in MCU module. 2. If parameter 'Wdg0SafetyEnable' is enabled then Wdg_17_Scu_SetMode and Wdg_17_Scu_SetTriggerCondition APIs are not applicable.		
SFRs Modified	WDTCPU0CON1		
Scope:	Driver		
Dependency:	None		

4.3.2 Wdg1SafetyEnable

Table 21 Wdg1SafetyEnable

Name:	Wdg1SafetyEnable		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_Cfg.h		
Range:	Values: STD_ON STD_OFF Default value: STD_OFF		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--
Description:	Used to enable safety features of WDG1 NOTE: If this parameter is enabled then user shall ensure that TOM/ATOM channels are not configured in MCU module.. This parameter is not applicable to EP devices. If parameter 'Wdg1SafetyEnable' is enabled then Wdg_17_Scu_SetMode and Wdg_17_Scu_SetTriggerCondition APIs are not applicable.		
SFRs Modified	WDTCPU1CON1		
Scope:	Driver		
Dependency:	None		

4.3.3 Wdg2SafetyEnable

Table 22 Wdg2SafetyEnable

Name:	Wdg2SafetyEnable		
Source:	This parameter is defined by IFX		
File:	NA		
Range:	Values: STD_ON		

	STD_OFF Default value: STD_OFF		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--
Description:	Used to enable safety features of WDG2. This is disabled for TC26x NOTE: If this parameter is enabled then user shall ensure that TOM/ATOM channels are not configured in MCU module. This parameter is not applicable to EP devices. If parameter 'Wdg2SafetyEnable' is enabled then Wdg_17_Scu_SetMode and Wdg_17_Scu_SetTriggerCondition APIs are not applicable.		
SFRs Modified	WDTCPU2CON1		
Scope:	Driver		
Dependency:	None		

4.3.4 WdgSetOffModeApi

Table 23 WdgSetOffModeApi

Name:	WdgSetOffModeApi		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_Cfg.h		
Range:	Values: STD_ON STD_OFF Default value: STD_OFF		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--
Description:	Used to enable Wdg_17_Scu_SetOffMode API		
SFRs Modified	WDTCPUxCON1		
Scope:	Driver		
Dependency:	None		

4.3.5 WdgEnableTimecheck

Table 24 WdgEnableTimecheck

Name:	WdgEnableTimecheck		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_Cfg.h		
Range:	Values: STD_ON STD_OFF Default value: STD_OFF		
Configuration	Pre-Compile	X	All variants

Class	Link Time	--	--
	Post Build	--	--
Description:	Used to enable WDG execution time check		
SFRs Modified	WDTCPuXCON1		
Scope:	Driver		
Dependency:	None		

4.4 Container: WdgDemEventParameterRefs

This container lists down all the Production error event configuration parameters for WDG Driver.

The multiplicity of this container ranges from 0 to 1.

If this container does not exist in the configuration, then DEM handling is not performed.

4.4.1 WdgDisableRejectedDemRef

Table 25 WDG_E_DISABLE_REJECTED

Name:	WdgDisableRejectedDemRef		
Source:	This parameter is as per AUTOSAR.		
File:	Wdg_17_Scu_Cfg.h		
Range:	Reference to "DemEventParameter"		
Configuration Class:	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	The existence of this parameter decides if the Wdg module would raise this particular dem. The user has to point to the right Dem event parameter for this fault. The short name of the pointed Dem parameter will be used as the symbol that will be passed as the first parameter to Dem_ReportEventStatus() function.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.2 WdgModeSwitchFailedDemRef

Table 26 WDG_E_MODE_FAILED

Name:	WdgModeSwitchFailedDemRef		
Source:	This parameter is as per AUTOSAR.		
File:	Wdg_17_Scu_Cfg.h		
Range:	Reference to "DemEventParameter"		
Configuration Class:	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--

Description:	The existence of this parameter decides if the Wdg module would raise this particular dem. The user has to point to the right Dem event parameter for this fault. The short name of the pointed Dem parameter will be used as the symbol that will be passed as the first parameter to Dem_ReportEventStatus() function.
SFRs Modified	None
Scope:	Driver
Dependency:	None

4.5 Container: WdgSettingsConfig

This container contains the configuration items for different Watchdog modes. Multiple instance of this container is possible only if WdgPBFixedAddress is FALSE.

4.5.1 WdgSystemClockRef

Table 27 WdgSystemClockRef

Name:	WdgSystemClockRef		
Source:	This parameter is microcontroller specific.		
File:	Wdg_17_Scu_Cfg.h		
Range:	Reference to a valid System Clock configured in MCU		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	-	-
	Post Build	-	-
SFRs Modified	None		
Scope:	Driver		
Dependency:	McuClockSettingConf		

4.5.2 WdgDefaultMode

Table 28 WdgDefaultMode

Name:	WdgDefaultMode		
Source:	This parameter is defined by AUTOSAR.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	WDGIF_OFF_MODE is possible if WdgDisableAllowed is STD_ON Values: WDGIF_FAST_MODE - Default Watchdog mode is fast for WDT0. WDGIF_SLOW_MODE - Default Watchdog mode is slow for WDT0. WDGIF_OFF_MODE - Default Watchdog mode is off for WDT0. Default value: WDGIF_FAST_MODE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
Description:	Default mode of Watchdog Driver initialization for WDT0. The Description on each Wdg		

	Mode is given below. 1. Off-Mode: The watchdog hardware is disabled / shut down. 2. Slow-Mode: Triggering the watchdog hardware can be done with a long timeout period. This mode can e.g. be used during system startup / initialization phase. 3. Fast-Mode: Triggering the watchdog hardware has to be done with a short timeout period. This mode can e.g. be used during normal operations of the ECU.
SFRs Modified	WDTCPU0CON1
Scope:	Driver
Dependency:	WdgDisableAllowed

4.5.3 Wdg1DefaultMode

Table 29 Wdg1DefaultMode

Name:	Wdg1DefaultMode		
Source:	This parameter is defined by IFX.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	WDGIF_OFF_MODE is possible if WdgDisableAllowed is STD_ON Values: WDGIF_FAST_MODE - Default Watchdog mode is fast for WDT1. WDGIF_SLOW_MODE - Default Watchdog mode is slow for WDT1. WDGIF_OFF_MODE - Default Watchdog mode is off for WDT1. Default value: WDGIF_FAST_MODE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
Description:	Default mode of Watchdog Driver initialization for WDT1. The Description on each Wdg Mode is given below. 1. Off-Mode: The watchdog hardware is disabled / shut down. 2. Slow-Mode: Triggering the watchdog hardware can be done with a long timeout period. This mode can e.g. be used during system startup / initialization phase. 3. Fast-Mode: Triggering the watchdog hardware has to be done with a short timeout period. This mode can e.g. be used during normal operations of the ECU. Note: This parameter is not applicable to EP devices.		
SFRs Modified	WDTCPU1CON1		
Scope:	Driver		
Dependency:	WdgDisableAllowed		

4.5.4 Wdg2DefaultMode

Table 30 Wdg2DefaultMode

Name:	Wdg2DefaultMode		
Source:	This parameter is defined by IFX.		
File:	Wdg_17_Scu_PBCfg.c		

Range:	WDGIF_OFF_MODE is possible if WdgDisableAllowed is STD_ON Values: WDGIF_FAST_MODE - Default Watchdog mode is fast for WDT2. WDGIF_SLOW_MODE - Default Watchdog mode is slow for WDT2. WDGIF_OFF_MODE - Default Watchdog mode is off for WDT2. Default value: WDGIF_FAST_MODE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
Description:	Default mode of Watchdog Driver initialization for WDT2. The Description on each Wdg Mode is given below. 1. Off-Mode: The watchdog hardware is disabled / shut down. 2. Slow-Mode: Triggering the watchdog hardware can be done with a long timeout period. This mode can e.g. be used during system startup / initialization phase. 3. Fast-Mode: Triggering the watchdog hardware has to be done with a short timeout period. This mode can e.g. be used during normal operations of the ECU. Note: This parameter is not applicable to EP devices		
SFRs Modified	WDTCPU2CON1		
Scope:	Driver		
Dependency:	WdgDisableAllowed		

4.5.5 Wdg0FastModeTimeoutValue

Table 31 Wdg0FastModeTimeoutValue

Name:	Wdg0FastModeTimeoutValue		
Source:	This parameter is defined by AUTOSAR under WdgSettingsFast container.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 0 limited by the equation $16777216 / F_{spb}(\text{min})$ Default value: 10		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	--	Variant PB
Description:	Fast mode timeout in milliseconds for WDT0.		
SFRs Modified	WDTCPU0CON0		
Scope:	Driver		
Dependency:	Value should be less than Wdg0SlowModeTimeoutValue. Value should be greater than Wdg0FastServiceGtmCbKTime.		

4.5.6 Wdg1FastModeTimeoutValue

Table 32 Wdg1FastModeTimeoutValue

Name:	Wdg1FastModeTimeoutValue		
Source:	This parameter is defined by AUTOSAR under WdgSettingsFast container.		
File:	Wdg_17_Scu_PBCfg.c		

Range:	Range: Values greater than 0 limited by the equation $16777216 / F_{spb}(\min)$ Default value: 10		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	--	Variant PB
Description:	Fast mode timeout in milliseconds for WDT1. Note: This parameter is not applicable to EP devices		
SFRs Modified	WDTCPU1CON0		
Scope:	Driver		
Dependency:	Value should be less than Wdg1SlowModeTimeoutValue. Value should be greater than Wdg1FastServiceGtmCbKTime.		

4.5.7 Wdg2FastModeTimeoutValue

Table 33 Wdg2FastModeTimeoutValue

Name:	Wdg2FastModeTimeoutValue		
Source:	This parameter is defined by AUTOSAR under WdgSettingsFast container.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 0 limited by the equation $16777216 / F_{spb}(\min)$ Default value: 10		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	--	Variant PB
Description:	Fast mode timeout in milliseconds for WDT2. Note: This parameter is not applicable to EP devices.		
SFRs Modified	WDTCPU2CON0		
Scope:	Driver		
Dependency:	Value should be less than Wdg2SlowModeTimeoutValue. Value should be greater than Wdg2FastServiceGtmCbKTime.		

4.5.8 Wdg0SlowModeTimeoutValue

Table 34 Wdg0SlowModeTimeoutValue

Name:	Wdg0SlowModeTimeoutValue		
Source:	This parameter is defined by AUTOSAR under WdgSettingsSlow container.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 0 limited by the equation $1073741824 / F_{spb}(\min)$ Default value: 20		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Slow mode timeout in Milliseconds for WDT0.		
SFRs Modified	WDTCPU0CON0		
Scope:	Driver		

Dependency:	Value should be greater than Wdg0SlowServiceGtmCbkTime.
--------------------	---

4.5.9 Wdg1SlowModeTimeoutValue

Table 35 Wdg1SlowModeTimeoutValue

Name:	Wdg1SlowModeTimeoutValue		
Source:	This parameter is defined by AUTOSAR under WdgSettingsSlow container.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 0 limited by the equation $1073741824 / F_{spb}(\min)$ Default value: 20		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Slow mode timeout in Milliseconds for WDT1. Note: This parameter is not applicable to EP devices		
SFRs Modified	WDTCPU1CON0		
Scope:	Driver		
Dependency:	Value should be greater than Wdg1SlowServiceGtmCbkTime.		

4.5.10 Wdg2SlowModeTimeoutValue

Table 36 Wdg2SlowModeTimeoutValue

Name:	Wdg2SlowModeTimeoutValue		
Source:	This parameter is defined by AUTOSAR under WdgSettingsSlow container.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 0 limited by the equation $1073741824 / F_{spb}(\min)$ Default value: 20		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Slow mode timeout in Milliseconds for WDT2. Note: This parameter is not applicable to EP devices.		
SFRs Modified	WDTCPU2CON0		
Scope:	Driver		
Dependency:	Value should be greater than Wdg0SlowServiceGtmCbkTime.		

Note: No Settings required for OFF Mode

4.5.11 Container: WdgInitialPassword

This container holds initial password for WDGx of CPUx respectively. This is enabled only if WdgxSafetyEnable, is true. Multiplicity 1:1

4.5.11.1 Wdg0InitialPassword

Table 37 Wdg0InitialPassword

Name:	Wdg0InitialPassword		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: 1 to 16383 Default value: 60		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Determines initial password of WDG0.		
SFRs Modified	WDTCPU0CON0		
Scope:	Driver		
Dependency:	Wdg0SafetyEnable		

4.5.11.2 Wdg1InitialPassword

Table 38 Wdg1InitialPassword

Name:	Wdg1InitialPassword		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: 1 to 16383 Default value: 60		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Determines initial password of WDG1. Note: This parameter is not applicable to EP devices.		
SFRs Modified	WDTCPU1CON0		
Scope:	Driver		
Dependency:	Wdg1SafetyEnable		

4.5.11.3 Wdg2InitialPassword

Table 39 Wdg2InitialPassword

Name:	Wdg2InitialPassword		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: 1 to 16383 Default value: 60		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Determines initial password of WDG2. Editable only if Wdg.MaxTimers > 2 Note: This parameter is not applicable to EP devices.		
SFRs Modified	WDTCPU2CON0		

Scope:	Driver
Dependency:	Wdg2SafetyEnable

4.5.12 Container: WdgTimerCheckTolerance

This container holds timer tolerance value for WDGx of CPUx respectively,. This is enabled only if WdgxSafetyEnable is true. Multiplicity 1:1

4.5.12.1 Wdg0TimerTolerance

Table 40 Wdg0TimerTolerance

Name:	Wdg0TimerTolerance		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: 0 to 127 Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Determines timer tolerance for WDG0.		
SFRs Modified	WDTCPU0CON1		
Scope:	Driver		
Dependency:	Wdg0SafetyEnable		

4.5.12.2 Wdg1TimerTolerance

Table 41 Wdg1TimerTolerance

Name:	Wdg1TimerTolerance		
Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: 0 to 127 Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Determines timer tolerance for WDG1. Note: This parameter is not applicable to EP devices.		
SFRs Modified	WDTCPU1CON1		
Scope:	Driver		
Dependency:	Wdg1SafetyEnable		

4.5.12.3 Wdg2TimerTolerance

Table 42 Wdg2TimerTolerance

Name:	Wdg2TimerTolerance
--------------	--------------------

Source:	This parameter is defined by IFX		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: 0 to 127 Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Determines timer tolerance of WDG2. This is disabled for TC26x. Note: This parameter is not applicable to EP devices		
SFRs Modified	WDTCPU2CON1		
Scope:	Driver		
Dependency:	Wdg2SafetyEnable		

4.6 Container: WdgExternalConfiguration

This container contains the configuration items for external watchdog hardware.

4.6.1 WdgExternalContainerRef

Table 43 WdgExternalContainerRef

Name:	WdgExternalContainerRef		
Source:	This parameter is defined by the AUTOSAR, refer 1 .		
File:	--		
Range:	--		
Configuration Class	Pre-Compile	X	All variants
	Link Time	--	--
	Post Build	--	--
Description:	This configuration is not provided as the external watchdog is not supported.		
SFRs Modified	None		
Scope:	--		
Dependency:	--		

4.7 Container: WdgGtmSettings

This container contains the configuration elements for configuring GTM hardware.

The settings here are used to configure the timing needs for Wdg module.

4.7.1 Wdg0ServiceGtmChannelRef

Table 44 Gtm channel reference to service watchdog hardware (WDT0)

Name:	Wdg0ServiceGtmChannelRef		
Source:	This parameter is microcontroller specific.		
File:	NA		
Range:	NA		
Configuration	Pre-Compile	--	--

Class	Link Time	--	--
	Post Build	X	
Description:	This references the particular channel of Gtm which is used to trigger the hardware WDT0. The module number and channel number obtained from the referred Gtm channel is used as the identity for all the Gtm APIs that gets called from Wdg (for servicing H/W WDT0). Note: While configuring the GTM channel for WDGx, make sure that interrupt SRV's for all WDG's are different. Configure IRQ such that, CPU0services the interrupt generated by GTM channel selected for WDG0		
SFRs Modified	ATOMi_AGC_ENDIS_STAT, TOMi_TGC0_ENDIS_STAT, TOMi_TGC1_ENDIS_STAT		
Scope:	Driver		
Dependency:	Gtm channel container under Gtm module.		

4.7.2 Wdg1ServiceGtmChannelRef

Table 45 Gtm channel reference to service watchdog hardware (WDT1)

Name:	Wdg1ServiceGtmChannelRef		
Source:	This parameter is microcontroller specific.		
File:	NA		
Range:	NA		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	
Description:	This references the particular channel of Gtm which is used to trigger the hardware WDT1. The module number and channel number obtained from the referred Gtm channel is used as the identity for all the Gtm APIs that gets called from Wdg (for servicing H/W WDT1). Note: While configuring the GTM channel for WDGx, make sure that interrupt SRV all 3 WDG's are different. Configure IRQ such that, CPU1 services the interrupt generated by GTM channel selected for WDG1 Note: This parameter is not applicable to EP devices		
SFRs Modified	ATOMi_AGC_ENDIS_STAT, TOMi_TGC0_ENDIS_STAT, TOMi_TGC1_ENDIS_STAT		
Scope:	Driver		
Dependency:	Gtm channel container under Gtm module.		

4.7.3 Wdg2ServiceGtmChannelRef

Table 46 Gtm channel reference to service watchdog hardware (WDT2)

Name:	Wdg2ServiceGtmChannelRef		
Source:	This parameter is microcontroller specific.		
File:	NA		
Range:	NA		

Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	
Description:	This references the particular channel of Gtm which is used to trigger the hardware WDT2. The module number and channel number obtained from the referred Gtm channel is used as the identity for all the Gtm APIs that gets called from Wdg (for servicing H/W WDT2). Note: While configuring the GTM channel for WDGx, make sure that interrupt SRV all 3 WDG's are different. Configure IRQ such that, CPU2 services the interrupt generated by GTM channel selected for WDG2 Note: This parameter is not applicable to EP devices		
SFRs Modified	ATOMi_AGC_ENDIS_STAT, TOMi_TGC0_ENDIS_STAT, TOMi_TGC1_ENDIS_STAT		
Scope:	Driver		
Dependency:	Gtm channel container under Gtm module.		

4.7.4 Wdg0SlowServiceGtmCbKTime

Table 47 Slow Service callback period of Gtm for WDT0

Name:	Wdg0SlowServiceGtmCbKTime		
Source:	This parameter is microcontroller specific.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Default: 15.0 Range: Values greater than 1.0 and Wdg0FastServiceGtmCbKTime and less than Wdg0SlowModeTimeoutValue.		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Slow mode Gtm callback period in Milliseconds for WDT0. $\text{GTM_Clock} = (\text{GTM_Clock} * (\text{GtmGlobalClkDivDenominator})) / (\text{GtmGlobalClkDivNumerator})$ if Wdg0ServiceGtmChannelRef selected as TOM channel if CmuFxdClkSourceSelect == "GTM_GLOBAL_CLOCK" $\text{GTM_Fixed_Clock} = \text{GTM_Clock}$ else $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = ((\text{GTM_Fixed_Clock}) / (2^{\text{Exponent}(4*n)})) / 1000 \text{ (GTM_FIXED_CLOCK_n where } n = 0 \text{ to } 4)$ True condition for selction of Wdg0SlowServiceGtmCbKTime : (Wdg0SlowServiceGtmCbKTime * TicksPerMs) value should be less than 65535 if Wdg0ServiceGtmChannelRef selected as ATOM channel $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = (\text{GTM_Fixed_Clock}) / 1000$ True condition for selction of Wdg0SlowServiceGtmCbKTime : (Wdg0SlowServiceGtmCbKTime * TicksPerMs) value should be less than 16777215		

SFRs Modified	TOMi_CHx_CM0, TOMi_CHx_SR0, ATOMi_CHx_CM0, ATOMi_CHx_SR0
Scope:	Driver
Dependency:	None

4.7.5 Wdg1SlowServiceGtmCbKTime

Table 48 Slow Service callback period of Gtm for WDT1

Name:	Wdg1SlowServiceGtmCbKTime		
Source:	This parameter is microcontroller specific.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Default: 15.0 Range: Values greater than 1.0 and Wdg1FastServiceGtmCbKTime and less than Wdg1SlowModeTimeoutValue.		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Slow mode Gtm callback period in Milliseconds for WDT1. Note: This parameter is not applicable to EP devices $\text{GTM_Clock} = (\text{GTM_Clock} * (\text{GtmGlobalClkDivDenominator})) / (\text{GtmGlobalClkDivNumerator})$ if Wdg1ServiceGtmChannelRef selected as TOM channel if CmuFxdClkSourceSelect == "GTM_GLOBAL_CLOCK" $\text{GTM_Fixed_Clock} = \text{GTM_Clock}$ else $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ TicksPerMs = ((GTM_Fixed_Clock) / (2 Exponet(4*n)))/1000 (GTM_FIXED_CLOCK_n where n = 0 to 4) True condition for selction of Wdg1SlowServiceGtmCbKTime : (Wdg1SlowServiceGtmCbKTime * TicksPerMs) value should be less than 65535 if Wdg1ServiceGtmChannelRef selected as ATOM channel $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ TicksPerMs = (GTM_Fixed_Clock) / 1000 True condition for selction of Wdg1SlowServiceGtmCbKTime : (Wdg1SlowServiceGtmCbKTime * TicksPerMs) value should be less than 16777215		
SFRs Modified	TOMi_CHx_CM0, TOMi_CHx_SR0, ATOMi_CHx_CM0, ATOMi_CHx_SR0		
Scope:	Driver		
Dependency:	None		

4.7.6 Wdg2SlowServiceGtmCbKTime

Table 49 Slow Service callback period of Gtm for WDT2

Name:	Wdg2SlowServiceGtmCbKTime
Source:	This parameter is microcontroller specific.

File:	Wdg_17_Scu_PBCfg.c		
Range:	Default: 15.0 Range: Values greater than 1.0 and Wdg2FastServiceGtmCbKTime and less than Wdg2SlowModeTimeoutValue.		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Slow mode Gtm callback period in Milliseconds for WDT2. Note: This parameter is not applicable to EP devices. $\text{GTM_Clock} = (\text{GTM_Clock} * (\text{GtmGlobalClkDivDenominator})) / (\text{GtmGlobalClkDivNumerator})$ if Wdg2ServiceGtmChannelRef selected as TOM channel if CmuFxdClkSourceSelect == "GTM_GLOBAL_CLOCK" GTM_Fixed_Clock = GTM_Clock else GTM_Fixed_Clock = GTM_Clock / (CmuConfigurableClknDiv+1) (where n = 0 to 7) TicksPerMs = ((GTM_Fixed_Clock) / (2 Exponet(4*n))) / 1000 (GTM_FIXED_CLOCK_n where n = 0 to 4) True condition for selction of Wdg2SlowServiceGtmCbKTime : (Wdg2SlowServiceGtmCbKTime * TicksPerMs) value should be less than 65535 if Wdg2ServiceGtmChannelRef selected as ATOM channel GTM_Fixed_Clock = GTM_Clock / (CmuConfigurableClknDiv+1) (where n = 0 to 7) TicksPerMs = (GTM_Fixed_Clock) / 1000 True condition for selction of Wdg2SlowServiceGtmCbKTime : (Wdg2SlowServiceGtmCbKTime * TicksPerMs) value should be less than 16777215		
SFRs Modified	TOMi_CHx_CM0, TOMi_CHx_SR0, ATOMi_CHx_CM0, ATOMi_CHx_SR0		
Scope:	Driver		
Dependency:	None		

4.7.7 Wdg0FastServiceGtmCbKTime

Table 50 Fast Service callback period of Gtm for WDT0

Name:	Wdg0FastServiceGtmCbKTime		
Source:	This parameter is microcontroller specific.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 1.0 and less than Wdg0FastModeTimeoutValue. Default: 7.0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Fast mode Gtm callback period in Milliseconds for WDT0. $\text{GTM_Clock} = (\text{GTM_Clock} * (\text{GtmGlobalClkDivDenominator})) / (\text{GtmGlobalClkDivNumerator})$ if Wdg0ServiceGtmChannelRef selected as TOM channel if CmuFxdClkSourceSelect == "GTM_GLOBAL_CLOCK"		

	$\text{GTM_Fixed_Clock} = \text{GTM_Clock}$ else $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = ((\text{GTM_Fixed_Clock}) / (2^{\text{Exponet}(4 \cdot n)})) / 1000$ (GTM_FIXED_CLOCK_n where n = 0 to 4) True condition for selction of Wdg0FastServiceGtmCbKTime : (Wdg0FastServiceGtmCbKTime * TicksPerMs) value should be less than 65535 if Wdg0ServiceGtmChannelRef selected as ATOM channel $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = (\text{GTM_Fixed_Clock}) / 1000$ True condition for selction of Wdg0FastServiceGtmCbKTime : (Wdg0FastServiceGtmCbKTime * TicksPerMs) value should be less than 16777215
SFRs Modified	TOMi_CHx_CM0, TOMi_CHx_SR0, ATOMi_CHx_CM0, ATOMi_CHx_SR0
Scope:	Driver
Dependency:	Value should be lesser than Wdg0SlowServiceGtmCbKTime

4.7.8 Wdg1FastServiceGtmCbKTime

Table 51 Fast Service callback period of Gtm for WDT1

Name:	Wdg1FastServiceGtmCbKTime		
Source:	This parameter is microcontroller specific.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 1.0 and less than Wdg1FastModeTimeoutValue. Default: 7.0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Fast mode Gtm callback period in Milliseconds for WDT1. Note: This parameter is not applicable to EP devices $\text{GTM_Clock} = (\text{GTM_Clock} * (\text{GtmGlobalClkDivDenominator})) / (\text{GtmGlobalClkDivNumerator})$ if Wdg1ServiceGtmChannelRef selected as TOM channel if CmuFxdClkSourceSelect == "GTM_GLOBAL_CLOCK" $\text{GTM_Fixed_Clock} = \text{GTM_Clock}$ else $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = ((\text{GTM_Fixed_Clock}) / (2^{\text{Exponet}(4 \cdot n)})) / 1000$ (GTM_FIXED_CLOCK_n where n = 0 to 4) True condition for selction of Wdg1FastServiceGtmCbKTime : (Wdg1FastServiceGtmCbKTime * TicksPerMs) value should be less than 65535 if Wdg1ServiceGtmChannelRef selected as ATOM channel $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = (\text{GTM_Fixed_Clock}) / 1000$ True condition for selction of Wdg1FastServiceGtmCbKTime : (Wdg1FastServiceGtmCbKTime * TicksPerMs) value should be less than 16777215		

SFRs Modified	TOMi_CHx_CM0, TOMi_CHx_SR0, ATOMi_CHx_CM0, ATOMi_CHx_SR0
Scope:	Driver
Dependency:	Value should be lesser than Wdg1SlowServiceGtmCbKTime

4.7.9 Wdg2FastServiceGtmCbKTime

Table 52 Fast Service callback period of Gtm for WDT2

Name:	Wdg2FastServiceGtmCbKTime		
Source:	This parameter is microcontroller specific.		
File:	Wdg_17_Scu_PBCfg.c		
Range:	Range: Values greater than 1.0 and less than Wdg2FastModeTimeoutValue. Default: 7.0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Fast mode Gtm callback period in Milliseconds for WDT2. Note: This parameter is not applicable to EP devices $\text{GTM_Clock} = (\text{GTM_Clock} * (\text{GtmGlobalClkDivDenominator})) / (\text{GtmGlobalClkDivNumerator})$ if Wdg2ServiceGtmChannelRef selected as TOM channel if CmuFxdClkSourceSelect == "GTM_GLOBAL_CLOCK" $\text{GTM_Fixed_Clock} = \text{GTM_Clock}$ else $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = ((\text{GTM_Fixed_Clock}) / (2^{\text{Exponent}(4*n)})) / 1000 \text{ (GTM_FIXED_CLOCK_n where } n = 0 \text{ to } 4)$ True condition for selection of Wdg2FastServiceGtmCbKTime : $(\text{Wdg2FastServiceGtmCbKTime} * \text{TicksPerMs})$ value should be less than 65535 if Wdg2ServiceGtmChannelRef selected as ATOM channel $\text{GTM_Fixed_Clock} = \text{GTM_Clock} / (\text{CmuConfigurableClknDiv} + 1) \text{ (where } n = 0 \text{ to } 7)$ $\text{TicksPerMs} = (\text{GTM_Fixed_Clock}) / 1000$ True condition for selection of Wdg2FastServiceGtmCbKTime : $(\text{Wdg2FastServiceGtmCbKTime} * \text{TicksPerMs})$ value should be less than 16777215		
SFRs Modified	TOMi_CHx_CM0, TOMi_CHx_SR0, ATOMi_CHx_CM0, ATOMi_CHx_SR0		
Scope:	Driver		
Dependency:	Value should be lesser than Wdg2SlowServiceGtmCbKTime		

Note: While configuring a TOM channel, make sure "TomChUpdateEnableOnCn0Reset" is checked. While configuring an ATOM channel make sure "AtomChUpdateEnableOnCn0Reset" is checked and "AtomChannelModeSelect" has a value "ATOM_SIG_OUTPUT_MODE_PWM". For other settings related to GTM, Refer [6](#).

4.8 Container:CommonPublishedInformation

4.8.1 ArMajorVersion

Table 53 ArMajorVersion

Name:	ArMajorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	2		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the major version of the Autosar Specification.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.2 ArMinorVersion

Table 54 ArMinorVersion

Name:	ArMinorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	5		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the minor version of the Autosar Specification.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.3 ArPatchVersion

Table 55 ArPatchVersion

Name:	ArPatchVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	0		

Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the patch version of the Autosar Specification.		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.4 SwMajorVersion

Table 56 SwMajorVersion

Name:	SwMajorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	Refer plugin.xml of the WDG module		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the major version of the Software Driver		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.5 SwMinorVersion

Table 57 SwMinorVersion

Name:	SwMinorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	Refer plugin.xml of the WDG module		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the minor version of the Software Driver		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.6 SwPatchVersion

Table 58 SwPatchVersion

Name:	SwPatchVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	Refer plugin.xml of the WDG module		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the patch version of the Software Driver		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.7 VendorId

Table 59 VendorId

Name:	VendorId		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	17		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the VendorId		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.8 ModuleId

Table 60 ModuleId

Name:	ModuleId		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	102		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the VendorId		

SFRs Modified	None
Scope:	Driver
Dependency:	None

4.8.9 Release

Table 61 Release

Name:	Release		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	_TRICORE_TC275 / _TRICORE_TC264/ _TRICORE_TC299		
Range:	NA		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
Description:	This parameter provides the Release ID		
SFRs Modified	None		
Scope:	Driver		
Dependency:	None		

4.9 Derived Configuration Parameters

These parameters represent the additional information required by the Driver and are automatically calculated by the tool

4.9.1 WDG_MAXSAFE_TIMERS

Table 62 WDG_MAXSAFE_TIMERS

Name:	WDG_MAXSAFE_TIMERS
Type:	#define
File:	Wdg_17_Scu_Cfg.h
Description:	This parameter is used to enable safety features of WDG
Range	0 to 2 – TC26x, 0 to 3 – TC27x, TC29x 0 to 1 – TC23x
Algorithm:	Number of WDGx with WdgxSafetyEnable = true
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	WdgxSafetyEnable

4.9.2 WDG_MAX_TIMERS

Table 63 WDG_MAX_TIMERS

Name:	WDG_MAX_TIMERS
--------------	----------------

Type:	#define
File:	Wdg_17_Scu_Cfg.h
Description:	This parameter is used to hold maximum number of WDG timers in device. Read from properties file of respective targets
Range	2 – TC26x, 3 – TC27x, TC29x 1 – TC23x
Algorithm:	none
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	none

4.9.3 WDG_MAXQM_TIMERS

Table 64 WDG_MAXQM_TIMERS

Name:	WDG_MAXQM_TIMERS
Type:	#define
File:	Wdg_17_Scu_Cfg.h
Description:	This parameter is used to enable safety features of WDG
Range	0 to 2 – TC26x, 0 to 3 – TC27x, TC29x 0 to 1 – TC23x
Algorithm:	WDG_MAX_TIMERS – WDG_MAXSAFE_TIMERS
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	WdgxSafetyEnable ,

NOTE These derived parameters are **precompile** parameters and hence the parameters in the “Dependency” field should not be modified after the build process.

4.9.4 WDG_RUNNING_IN_USER_0_MODE_ENABLE

Table 65 WDG_RUNNING_IN_USER_0_MODE_ENABLE

Name:	WDG_RUNNING_IN_USER_0_MODE_ENABLE
Type:	#define
File:	Wdg_17_Scu_Cfg.h
Description:	This parameter indicates Running in user mode 0 is enabled or not.
Range	STD_ON – User 0 mode is enabled for API's based on selection STD_OFF– User 1/ supervisory mode is enabled for API's based on selection
Algorithm:	1.get the user 0 mode enable value using the xpath variable \$WdgRunningInUser0Mode 2. Give an error if none of the switches \$WdgUserModeInitApiEnable or \$WdgUserModeRuntimeApiEnable are not set to true and the variable

Name:	WDG_RUNNING_IN_USER_0_MODE_ENABLE
Type:	#define
File:	Wdg_17_Scu_Cfg.h
	\$WdgRunningInUser0Mode is true 3. If there is no error then generate the value of node \$WdgRunningInUser0Mode
Design Decisions:	If the switch is on then any of the API's should be selected to run in User Mode 0
Asserts/Warnings/Info Raised:	When WdgGeneralConfiguration/WdgUserModelInitApiEnable is set as true, Then atleast any one of the switches: WdgUserModelInitApiEnable or WdgUserModeRuntimeApiEnable should be selected.
Dependencies	WdgUserModelInitApiEnable or WdgUserModeRuntimeApiEnable

4.9.5 WDG_USER_MODE_INIT_API_ENABLE

Table 66 WDG_USER_MODE_INIT_API_ENABLE

Name:	WDG_USER_MODE_INIT_API_ENABLE
Type:	#define
File:	Wdg_17_Scu_Cfg.h
Description:	This parameter indicates Init API is run in user mode or not. If this switch is ON then INIT API is run in user mode else if it is OFF it is run in supervisory mode
Range	STD_ON – Init API is run in User Mode STD_OFF– Init API is run in supervisory mode
Algorithm:	1.Get the Init Api Enable value using the xpath variable \$WdgUserModelInitApiEnable and print.
Design Decisions:	If the switch is on then any of the API's should be selected to run in User Mode 0
Asserts/Warnings/Info Raised:	
Dependencies	WdgRunningInUser0Mode

4.9.6 WDG_USER_MODE_RUNTIME_API_ENABLE

Table 67 WDG_USER_MODE_RUNTIME_API_ENABLE

Name:	WDG_U Wdg_17_Scu_Cfg.h SER_MODE_RUNTIME_API_ENABLE
Type:	#define
File:	Wdg_17_Scu_Cfg.h
Description:	This parameter indicates all API other than init and deinit are run in user mode or not. If this switch is ON then all APIs other than Init and Delnit are run in user mode else if it is OFF it is run in supervisory mode
Range	STD_ON –APIs are run in User Mode

Name:	WDG_U Wdg_17_Scu_Cfg.h SER_MODE_RUNTIME_API_ENABLE
Type:	#define
File:	Wdg_17_Scu_Cfg.h
	STD_OFF– APIs are run in supervisory mode
Algorithm:	1.Get the Runtime Api Enable value using the xpath variable \$ WdgUserModeRuntimeApiEnableand print.
Design Decisions:	The selection between user mode 0 and user mode 1 is taken by switch WdgRunningInUser0Mode. If the switch is ON API will run in User 0 mode else it will run in USER 1 Mode if the WdgUserModeRuntimeApiEnable switch is ON.
Asserts/Warnings/Info Raised:	-
Dependencies	WdgRunningInUser0Mode

4.10 Property File Variables

4.10.1 Wdg.MaxTimers

Table 68 Wdg.MaxTimers

Name:	Wdg.MaxTimers
Type:	Property File Variable
File:	Aurix_<Derivative>.properties
Description:	This parameter specifies the number of WDG timers.
Range	2 -TC26x 3 – TC27x, TC29x 1 – TC23x
Algorithm:	None
Design Decisions:	None
Asserts/Warnings/Info Raised:	None
Dependencies	None

5 Published parameters

This section describes the parameters published by the WDG Driver.

5.1 WDG Trigger Mode

Table 69 WDG Trigger Mode

Name:	WDG_TRIGGER_MODE
Type:	#define
File:	Wdg_17_Scu_Cfg.h
Value:	WDG_TOGGLE
Description:	Trigger mode tells about the supported mode of WDT operation in AURIX microcontrollers.

5.2 VendorId

Table 70 VendorId

Syntax:	WDG_17_SCU_VENDOR_ID
Type:	Macro - #define
File:	Wdg_17_Scu.h
Value:	17
Description:	Vendor is Infineon Technologies

5.3 ModuleId

Table 71 ModuleId

Syntax:	WDG_17_SCU_MODULE_ID
Type:	Macro - #define, type casted to uint16
File:	Wdg_17_Scu.h
Value:	101 (0x65)
Description:	This macro gives the Wdg Driver module ID as described by AUTOSAR

5.4 SwMajorVersion

Table 72 SwMajorVersion

Syntax:	WDG_17_SW_MAJOR_VERSION
Type:	Macro - #define
File:	Wdg_17_Scu_Cfg.h
Value:	Refer plugin.xml of the WDG module
Description:	Major version number of the vendor specific implementation of the Driver. The numbering is vendor specific. The MAJOR_VERSION is incremented if the Driver is not downwards compatible any more (e.g. existing API changed)

5.5 SwMinorVersion

Table 73 SwMinorVersion

Syntax:	WDG_17_SCU_SW_MINOR_VERSION
Type:	Macro - #define
File:	Wdg_17_Scu_Cfg.h
Value:	Refer plugin.xml of the WDG module
Description:	Minor version number of the vendor specific implementation of the Driver. The numbering is vendor specific. MINOR_VERSION is incremented if the Driver is still downwards compatible (e.g. new functionality added)

5.6 SwPatchVersion

Table 74 SwPatchVersion

Syntax:	WDG_17_SCU_SW_PATCH_VERSION
Type:	Macro - #define
File:	Wdg_17_Scu_Cfg.h
Value:	Refer plugin.xml of the WDG module
Description:	Patch level version number of the vendor specific implementation of the Driver. The numbering is vendor specific. The PATCH_VERSION is incremented if the Driver is still upwards and downwards compatible (e.g. bug fixed)

5.7 ARMajorVersion

Table 75 ARMajorVersion

Syntax:	WDG_17_SCU_AR_RELEASE_MAJOR_VERSION
Type:	Macro - #define
File:	Wdg_17_Scu_Cfg.h
Value:	4
Description:	Major version number of the AUTOSAR specification.

5.8 ARMinorVersion

Table 76 ARMinorVersion

Syntax:	WDG_17_SCU_AR_RELEASE_MINOR_VERSION
Type:	Macro - #define
File:	Wdg_17_Scu_Cfg.h
Value:	0
Description:	Minor version number of the AUTOSAR specification.

5.9 ARPatchVersion

Table 77 ARPatchVersion

CONFIDENTIAL**Published parameters**

Syntax:	WDG_17_SCU_AR_RELEASE_REVISION_VERSION
Type:	Macro - #define
File:	Wdg_17_Scu_Cfg.h
Value:	3
Description:	Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

6 API Documentation

6.1 API Type Definitions

6.1.1 Type Definition Wdg_ConfigType

Table 78 Type Definition Wdg_ConfigType

Syntax:	Wdg_ConfigType
Type:	struct
File:	Wdg_17_Scu.h
Range:	This data type is used for the configuration of the entire WDG Driver.
Description:	The configuration structure for initializing WDG. The Pointer to the object of this type is passed via Wdg_17_Scu_Init API to initialize the WDG Driver.

6.1.2 Type Definition Wdg_StatusType

Table 79 Type Definition Wdg_StatusType

Syntax:	WdgIf_StatusType
Type:	enum
File:	Wdg_Type.h
Range:	WDG_UNINIT: Driver un-initialized WDG_IDLE: Driver is in Idle state WDG_BUSY: Driver busy in executing an API of WDG.
Description:	This type defines a range of specific Driver status for WDG. Used when Development Error Detection is enabled.

6.2 API Functions

This chapter describes the API interfaces provided by WDG Driver.

6.2.1 Wdg_17_Scu_Init

Table 80 Specification of API Wdg_17_Scu_Init

Service name:	Wdg_17_Scu_Init	
Syntax:	void Wdg_17_Scu_Init(const Wdg_ConfigType *ConfigPtr)	
Service ID:	0x00	
Sync/ Async:	Synchronous	
Re-entrancy:	Non-reentrant	
Parameters (in):	ConfigPtr	Pointer to WDG configuration Set
Parameters (out):	None	
Return value:	None	
Description:	If WDTx initialized without safety.	

	This routine initializes the Watchdog driver and Watchdog hardware, i.e. it sets the default Watchdog mode and initial timeout period as provided in the configuration set. The routine will initialize all Driver global variables and those controller registers that are needed for controlling the Watchdog hardware. The Watchdog hardware will be running after this routine succeeded. All timeout periods will have the same (configured) duration, i.e. the first timeout period will have the same duration as the following timeout periods. If development error detection is enabled, the routine will set the drivers internal state from WDG_UNINIT (the default state) to WDG_IDLE if the initialization was successful. It will configure the GTM timer with Slow or Fast call back rate based on the initial WDG Mode. If WDTx to be initialized with safety: This routine enables WDGx with safety features. Enables password sequencing and execution time check. Initial password and timer tolerance values are set from configuration. Reload value is set as per the mode selected. DEM is raised if safety features are not initialized correctly All WDG's can be initialized through this API. If the API is invoked from CPU COREx, then WDGx is initialized. Note: Make sure the same ConfigPtr is passed for all WDG's, while initializing through multiple cores. ConfigPtr cannot change across cores.
Caveats:	For WDGx with safety Wdg_17_Scu_Init() shall be the last function to be called and all calls to Mcal_ResetENDINIT()/Mcal_SetENDINIT() shall be before WDG initialization
Configuration:	The WDG default mode and the reload values for fast and slow modes shall be configured by the configuration tool.
DET:	This DET is applicable only if Fixed Address feature is FALSE WDG_E_PARAM_CONFIG is reported when passed ConfigPtr parameter is NULL pointer or any Hardware related parameters is out of range in the configuration
DEM:	WDG_E_DISABLE_REJECTED is reported if WDG_DISABLE_ALLOWED is defined as false ('0') but requested default mode is WDGIF_OFF_MODE. WDG_E_MODE_FAILED is reported if WDG enable with safety fails. WDG_E_INIT_FAILED is reported if Automatic password Sequencing or Time Checking is enabled before calling Wdg_17_Scu_Init() API.
Implementation Comments:	1. Same config root shall be used to initialize all 3 WDG's 2. If WDTx is to be initialized with safety enabled then initial mode shall be SLOW/FAST. 3. Password Sequencing and Time Checking for the WDT should be disabled before calling the API, to avoid possible DEM.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter WdgRunningInUser0Mode is 'True' and WdgUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModelInitApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModelInitApiEnable is 'False'.

6.2.2 Wdg_17_Scu_SetMode

Table 81 Specification of API Wdg_17_Scu_SetMode

Service name:	Wdg_17_Scu_SetMode
Syntax:	Std_ReturnType Wdg_17_Scu_SetMode(WdgIf_ModeType Mode)
Service ID:	0x01
Sync/ Async:	Synchronous
Re-entrancy:	Non-reentrant
Parameters (in):	One of the following statically configured modes: WDGIF_OFF_MODE, WDGIF_SLOW_MODE, WDGIF_FAST_MODE
Parameters (out):	None
Return value:	Std_ReturnType : E_OK if the function succeeded i.e. the mode switch has been completely executed, E_NOT_OK otherwise.
Description:	By choosing one of a limited number of statically configured settings (different timeout periods) the Watchdog driver and hardware can be switched among the following three different Watchdog modes: WDGIF_OFF_MODE, WDGIF_SLOW_MODE, WDGIF_FAST_MODE The parameters needed for the different Watchdog modes are hardware / driver specific and contained in the Watchdog driver's configuration set provided in the initialization routine. The routine will switch the Watchdog driver and hardware from the current Watchdog mode to the Watchdog mode defined by the given parameter. It will return E_OK if this mode switch has been executed completely and successfully, i.e. all parameters of the Watchdog driver and hardware have been set to the new values. If switching the Watchdog driver and hardware into the requested mode is not possible, e.g. because of inconsistent mode settings or because some timing constraints have not been met, the routine will return the value E_NOT_OK. This error will be reported to the Diagnostic Event Manager (DEM) with the error code WDG_E_MODE_SWITCH_FAILED. Mode for all WDG's can be changes through this API. If the API is invoked from CPU COREx then, mode for WDGx is changed. If development error detection is enabled, the routine will set the driver state to WDG_BUSY during its runtime. Note: This API is visible when Watchdog timer is configured for QM mode. API is available for use, if parameter 'WdgxSafetyEnable' is set to FALSE. (where x = 0 1,2,3 and for EP x =0) The WDG in Safety Mode is not compatible with the Autosar WDG – integration In AS BSW needs also adaptation of WDGIF.
Caveats:	None
Configuration:	None

DET:	DET error WDG_E_DRIVER_STATE is reported if the Driver state is not WDGIF_IDLE (i.e. driver not initialized or driver initialized but busy executing another Wdg API). DET error WDG_E_PARAM_MODE is reported if the requested mode change is not one of the following three different Watchdog modes: WDGIF_OFF_MODE, WDGIF_SLOW_MODE, WDGIF_FAST_MODE
DEM:	WDG_E_DISABLE_REJECTED is reported if WDG_DISABLE_ALLOWED is defined as false ('0') but requested mode change is WDGIF_OFF_MODE. WDG_E_MODE_SWITCH_FAILED is reported if WDG fails to switch to the given mode.
Implementation Comments:	This function starts the Gtm channel assigned to SRV window. When there is a transition from WDGIF_OFF_MODE to any of the other two modes, user should call the Wdg_17_Scu_SetTriggerCondition immediately in order to prevent WDG from overflow and trigger the SMU. SMU will respond based on its configuration (if reset is configured it will reset the system). Calling this function has no effect if the parameter Wdg_TimeoutCounter has already become zero. User shall call this function before the parameter Wdg_TimeoutCounter becomes zero.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter WdgRunningInUser0Mode is 'True' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'False'.

6.2.3 Wdg_17_Scu_SetTriggerCondition

Table 82 Specification of API Wdg_17_Scu_SetTriggerCondition

Service name:	Wdg_17_Scu_SetTriggerCondition	
Syntax:	void Wdg_17_Scu_SetTriggerCondition(uint16 timeout)	
Service ID:	0x03	
Sync/ Async:	Synchronous	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	This routine takes care of self-triggering the Watchdog hardware (WDG) for the next "timeout" milliseconds. Before running out of "timeout" period, this API has to be called cyclically by some upper layer function (usually the Watchdog manager) in order to prevent the WDG from expiring. A "timeout" value of 0 passed to this function creates an almost instant triggering of SMU. SMU will respond based on its configuration (if reset is configured it will reset the system).	

	Timeout for all WDG's can be changed through this API. If the API is invoked from CPU COREx then, timeout for WDGx is changed. If development error detection is enabled, the routine will set the driver state to WDG_BUSY during its runtime. Note: This API is visible when Watchdog timer is configured for QM mode. API is available for use, if parameter 'WdgxSafetyEnable' is set to FALSE. (where x = 0 1,2,3 and for EP x =0) The WDG in Safety Mode is not compatible with the Autosar WDG – integration In AS BSW needs also adaptation of WDGIF.
Caveats:	The Watchdog driver would have been initialized before this service is called This routine might be called on interrupt level.
Configuration:	None
DET:	DET error WDG_E_DRIVER_STATE is reported if Driver state is not WDGIF_IDLE (i.e. driver not initialized or driver initialized but busy executing another Wdg API). Similary, when the "timeout" value passed is greater than the configured limit "WdgMaxTimeout", the DET WDG_E_PARAM_TIMEOUT is raised.
DEM:	None
Implementation Comments:	This function starts the Gtm channel assigned to STC window. Note: 1. When Wdg is in OFF mode, if this API is called, it is treated as an illegal behavior by the upper module and the Wdg module will create a reset state in the subsequent call to Wdg_17_Scu_SetMode (FAST/SLOW). 2. Ensure that this API is called immediately after a mode transition from OFF to SLOW/FAST. If not, this state would again be interpreted as a reset state.
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter WdgRunningInUser0Mode is 'True' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'False'.

6.2.4 Wdg_17_Scu_SafeTrigger

Table 83 Specification of API Wdg_17_Scu_SafeTrigger

Service name:	Wdg_17_Scu_SafeTrigger	
Syntax:	<pre>void Wdg_17_Scu_SafeTrigger (uint16 SeqPassword, uint16 TimerCheckValue)</pre>	
Service ID:	0x06	
Sync/ Async:	Synchronous	
Re-entrancy:	Non-reentrant	
Parameters (in):	uint16	Password to perform password access

	SeqPassword	(unlocking of WDTxCON0)
	uint16 TimerCheckValue	Timer value to be written into reload field while password access (if WDG_ENABLE_TIME_CHECK is STD_ON)
Parameters (out):	None	
Return value:	None	
Description:	Triggers the watchdog hardware. It has to be called cyclically by some upper layer function (usually the watchdog manager) in order to prevent the watchdog hardware from expiring. To trigger WDTx and prevent it from overflowing, the API must be called from corresponding CPU core. WDG servicing will include password access followed by modify access. During Password access, if execution time check is enabled then reload value will be written with TimerCheckValue else current reload value is written back to unlock WDTxCON0. Sequence password along will be used to unlock WDTxCON0 register. Wrong password access will set an SMU alarm by WDG hardware NOTE : User is expected to give the 14bit value of the password field. This value is shifted twice and written into PW field of CON1 register. NOTE: This API is visible when Watchdog timer is configured for safety (non AUTOSAR) mode.	
Caveats:	WDGx_SAFETY_ENABLE, shall be STD_ON to call this API	
Configuration:	WdgTimerTolerance, WdgSafetyEnable, WdgEnableTimecheck, WdgDefaultMode, Wdg1DefaultMode, Wdg2DefaultMode, WdgFastModeTimeoutValue, WdgSlowModeTimeoutValue	
DET:	None	
DEM:	None	
Implementation Comments:	WDG_MAXSAFE_TIMERS > 0 This API implementation is applicable only when selected Watchdog Timer is enabled with safety feature.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter WdgRunningInUser0Mode is 'True' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'False'.	

6.2.5 Wdg_17_Scu_SetOffMode

Table 84 Specification of API Wdg_17_Scu_SetOffMode

Service name:	Wdg_17_Scu_SetOffMode
Syntax:	<pre>void Wdg_17_Scu_SetOffMode (uint16 SeqPassword1, uint16 SeqPassword2)</pre>
Service ID:	0x07

Sync/ Async:	Synchronous	
Re-entrancy:	Non-reentrant	
Parameters (in):	uint16 Seqpassword1	Password to perform password access (unlocking of WDTxCON0,) to reset ENDINIT
	uint16 SeqPassword2	Password to perform password access (unlocking of WDTxCON0) to set ENDINIT. This parameter is derived from Seqpassword1
Parameters (out):	None	
Return value:	void	
Description:	Switches the WDTx into the OFF Mode and disable safety feature. This API must be called from CPUx (corresponding CPU core). Wdg_17_Scu_SetOffMode() requires two consecutive passwords (SeqPassword1,SeqPassword2) as input parameters. SeqPassword1 is used to unlock WDTxCON0 and reset ENDINIT bit. Set DR = 0, PAR = 0, TCR = 0,UR = 0 in WDTxCON1 register SeqPassword2 is used to unlock WDTxCON0 and set ENDINIT bit. WDTxSR register is updated with corresponding bit state. . Wrong password access sets an SMU alarm by WDG hardware Note: This API is visible when Watchdog timer is configured for Safety mode.	
Caveats:	WDG_SET_OFF_MODE_API shall be STD_ON	
Configuration:	WdgSetOffModeApi, WdgSafetyEnable	
DET:	None	
DEM:	None	
Implementation Comments:	WDG_MAXSAFE_TIMERS > 0 This API implementation is applicable only when selected Watchdog Timer is enabled with safety feature.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter WdgRunningInUser0Mode is 'True' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'False'.	

6.2.6 Wdg_17_Scu_InitAltResetPassword

Table 85 Specification of API Wdg_17_Scu_InitAltResetPassword

Service name:	Wdg_17_Scu_InitAltResetPassword
Syntax:	<pre>void Wdg_17_Scu_InitAltResetPassword (const Wdg_17_Scu_ConfigType *ConfigPtr, uint16 ResetPassword)</pre>
Service ID:	0x08
Sync/ Async:	Synchronous

Re-entrancy:	Non-reentrant	
Parameters (in):	ConfigPtr	Pointer to configuration set.
	ResetPassword	New initial password
Parameters (out):	None	
Description:	This routine enables WDGx with safety features. Enables password sequencing and execution time check if selected by configuration. Timer tolerance values are set from configuration but initial password is updated from the parameter ResetPassword, bypassing the configured initial password. Reload value is set as per the mode selected. DEM is raised if safety features are not initialized correctly. If this API is called for WDGx which is not safety enabled DET is reported. Note: Make sure the same ConfigPtr is passed for all WDG's, while initializing through multiple cores. ConfigPtr cannot change across cores. This API shall be called for safety enabled watchdogs only. Note: This API is visible when Watchdog timer is configured for safety (non AUTOSAR) mode.	
Caveats:	Configuration parameter WdgxSafetyEnable (x = 0,1,2), shall be STD_ON to call this API for corresponding watchdog. For safety enabled watchdog, the behavior is same as Wdg_17_Scu_Init other than the initial password value set. This API is provided to user, for cases where user wishes to start a new program flow check with an alternate initial password while software is executing. In such scenario user has to switch off the watchdog WDGx with SetOffMode API, and then use this API to initialize watchdog with new initial password (reset password).	
Configuration:	The WDG default mode and the reload values for fast and slow modes shall be configured by the configuration tool.	
DET:	WDG_E_PARAM_CONFIG is reported when passed ConfigPtr parameter is NULL pointer or any Hardware related parameters is out of range in the configuration, if Fixed Address feature is FALSE The WDG_E_PARAM_CONFIG is also reported if the API is called from Core whose watchdog is not safety enabled. The password in the hardware is 14 bit long, if the parameter ResetPassword is greater than 0x3FFF, development error WDG_E_PARAM_PASSWD is reported.	
DEM:	WDG_E_MODE_FAILED is reported if WDG enable with safety fails. WDG_E_INIT_FAILED is reported if Automatic password Sequencing(PAS) or Time Checking(TCS) is enabled before calling Wdg_17_Scu_InitAltResetPassword() API.	
Implementation Comments:	1. Same config root shall be used to initialize all 3 WDG's 2. The initial mode shall be SLOW/FAST. 3. Password Sequencing and Time Checking for the WDT should be disabled before calling the API, to avoid possible DEM.	
I/O Mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter WdgRunningInUser0Mode is 'True' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'False'.	

6.2.7 Wdg_17_Scu_GetVersionInfo

Table 86 Specification of API Wdg_17_Scu_GetVersionInfo

Service name:	Wdg_17_Scu_GetVersionInfo	
Syntax:	Wdg_17_Scu_GetVersionInfo (Std_VersionInfoType *versioninfo)	
Service ID:	0x04	
Sync/ Async:	Synchronous	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	
Parameters (out):	VersionInfoPtr	Pointer to store the version information of this driver.
Return value:	None	
Description:	This service returns the version information of this Driver. The version information includes: - Module Id - Vendor Id - SW Major Version - SW Minor Version - Instance Id - Vendor specific version numbers This function is pre compile time configurable On/Off by the configuration parameter: WDG_VERSION_INFO_API	
Caveats:	None	
Configuration:	This function is available only if the compile switch WDG_VERSION_INFO_API is TRUE	
DET:	DET error WDG_E_DRIVER_STATE is reported if the Driver state is WDGIF_UNINIT (i.e. driver not initialized).	
DEM:	None	
Implementation Comments:	This is implemented as a macro.	
I/O Mode:	This API shall be invoked when the CPU is in either User-0 or User-1 or Supervisor Mode.	

6.3 Callback Functions

Gtm provides the time base for all of Wdg's functionality. Wdg needs 1 Gtm channels to be configured with callback directed towards Wdg. They are as given below.

6.3.1 Wdg_17_Scu_Cbk_GtmNotifySRV

Table 87 Specification of callback Wdg_17_Scu_Cbk_GtmNotifySRV

Service name:	Wdg_17_Scu_Cbk_GtmNotifySRV	
Syntax:	void Wdg_17_Scu_Cbk_GtmNotifySRV(uint8 ModuleType, uint8 ModuleNo, uint8 ChannelNo, uint16 IrqNotifVal)	
Service ID:	NA	
Sync/ Async:	Synchronous	
Re-entrancy:	Non-reentrant	
Parameters (in):	None	

Parameters (out):	None
Return value:	None
Description:	This callback is used for servicing the wdg hardware.
Caveats:	None
Configuration:	Please configure WdgServiceGtmChannelRef to point to the right Gtm channel and ensure that the name of the callback configured in Gtm is the same as that of this callback. While configuring the GTM – TOM/ATOM channel, make sure TomChUpdateEnableOnCn0Reset/AtomChUpdateEnableOnCn0Reset is checked. Additionally, it has to be ensured that the corresponding Gtm channel is set in GTM_INTERRUPT_PULSE_NOTIFY_MODE or GTM_INTERRUPT_LEVEL_MODE. Pulse Notify interrupt mode - Each interrupt event will be signalled as a pulse to the Interrupt Router module. Level interrupt mode - Each interrupt event will be signalled as a constant signal level to the Interrupt router module. Note: If the interrupt timings are too small and due to some higher priority interrupts if the second interrupt event occurs when the first interrupt is being serviced, there is chance of missing the interrupt if level interrupt mode is used and hence the service of the WDG due to constant high signal for Interrupt Router unit. But if pulse-notify mode is used such interrupts and WDG services will not be lost. Hence to avoid such scenarios Pulse Notify mode is recommended. This API is called from the ISR of GTM. Same call back API is used for all the 3 WDG's. It is driver's responsibility to service the correct WDG. Note: The user shall take care that the GTM ISR for WDGx is serviced by CPU COREx only. IRQ module shall be configured correctly for GTM interrupts to be serviced by correct core
DET:	None
DEM:	None
Implementation Comments:	None
I/O mode:	This API shall be invoked when the CPU is in User-0 Mode provided configuration parameter WdgRunningInUser0Mode is 'True' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in User-1 Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'True'. This API shall be invoked when the CPU is in Supervisor Mode provided configuration parameter WdgRunningInUser0Mode is 'False' and WdgUserModeRuntimeApiEnable is 'False'.

6.4 Interrupt Handling

No interrupts are handled by WDG Driver.

6.5 Exclusive Area (EA) Information

The exclusive areas (critical sections) are provided for any of the following reason:

- 1) To protect shared resources from the module interrupts. The resources are used in ISR's.
- 2) To protect shared resources from another API / same API.

6.5.1 Wdg_Trigger

Table 88 Specification of Wdg_Trigger

API Names:	Wdg_LTrigger
ISR Name:	Wdg_17_Scu_Cbk_GtmNotifySRV
Comment	When Gtm callback invokes this function, this critical section is entered. If the WDTCPuXCON0 register is accessed by any other API/function then WDTCPuXCON0 register value will be corrupted. For ex: API's like Mcal_ResetENDINIT, Mcal_SetENDINIT. To avoid that critical section is introduced.

6.5.2 Wdg_TimerHandling

Table 89 Specification of Wdg_TimerHandling

API Names:	All timer related APIs/Local functions
ISR Name:	NA
Comment	This ensures consistency of the timer v.z Gtm(x) Where Gtm(x) is used for Servicing of watchdog hardware. This critical section is used to protect the Gtm related logic.

6.5.3 Wdg_ChangeMode

Table 90 Specification of Wdg_ChangeMode

API Names:	Wdg_LChangeMode
ISR Name:	NA
Comment	In order to protect the program flow of IChangeMode. If the WDTCPuXCON0 register is accessed by any other API/function then WDTCPuXCON0 register value will be corrupted. For ex: API's like Mcal_ResetENDINIT, Mcal_SetENDINIT.

7 Data Description

The below table lists all the global and static variables used in the WatchDog Driver.

Table 91 Global Data Description

Parameter	Type/Range	Description
Wdg_DriverState[WDG_MAX_TIMERS]	WdgIf_StatusType	To store WDG Driver State This variable and its type is required only if DET is enabled.
Wdg_KConfigPtr	static const Wdg_17_Scu_ConfigType *	To store the Wdg Driver configuration pointer. It is a common variable for all the cores.
Wdg_DriverMode[WDG_MAX_TIMERS]	WdgIf_ModeType	To store WDG Driver mode
Wdg_TimeoutCounter[WDG_MAXQM_TIMERS]	uint32	Holds the value of timeout counter.
Wdg_CurrentReloadValue[WDG_MAX_TIMERS]	uint16	Reload value of the current mode

8 Error classification

Table 92 Error Classification

Error Code	Error	Relevance	Value (hex)
WDG_E_DRIVER_STATE	API service used in wrong context (e.g. Driver not initialized)	Development	0x10
WDG_E_PARAM_MODE	API service called with wrong / inconsistent parameter(s)	Development	0x11
WDG_E_PARAM_CONFIG	API service called with wrong / inconsistent parameter(s)	Development	0x12
WDG_E_PARAM_PASSWD	Password provided is not in range	Development	0x15
WDG_E_PARAM_TIMEOUT	passed timeout value is higher than the maximum timeout value	Development	0x13
WDG_E_MODE_FAILED	Switching between Watchdog modes failed	Production	Assigned by DEM
WDG_E_DISABLE_REJECTED	Disabling of Watchdog not allowed (e.g. in safety relevant systems)	Production	Assigned by DEM
WDG_E_INIT_FAILED	APIs Wdg_17_Scu_Init() and Wdg_17_Scu_InitAltResetPassword() is called when Password Sequencing(PAS) or time checking(TCS) is enabled	Production	Assigned by DEM

There are no additional errors implemented corresponding to specific hardware properties.

9 Example Usage

This chapter describes how to use different API provided by WDG Driver.

9.1 Configuration of Driver

Configuration of other Drivers:

Configuration of System Clock: This Configuration is done using the MCU Driver.

WDG Configuration

- WDG General configuration
- WDG safety configuration
- WDG Mode Configuration

Refer to the demo configuration in the code package for an example configuration.

Note: WDG0 is configured as QM driver for an EP device TC23x.

9.2 Watchdog Operation

9.2.1 Watchdog Operation

WDGx can be configured with safety or without safety.

The WDG configured without safety uses window concept in AS403. The WDG module will be triggered by a GTM channel. The application has to set a window for which the GTM channel will trigger the WDG using the API `Wdg_17_Scu_SetTriggerCondition()`.

The WDG configured with safety does not use GTM channel for triggering (windowed triggering not supported) instead user has to trigger WDG before time out by calling `Wdg_17_Scu_SafeTrigger()`.

In the demo configuration WDG0 and WDG2 are configured without safety and WDG1 is configured with safety. For an EP device WDG0 is configured as without safety.

Following Steps should be taken:

- The WDG module has to be initialized using the `Wdg_17_Scu_Init ()` API. This will start the WDG in the Initial mode configured and initial timeout window. WDG2 initialization can be done by calling `Wdg_17_Scu_Init` from CPU Core 2 respectively. This initialization shall be done only after MCU and WDG initialization for Core 0 is completed.
- In demo configuration WDG0 and WDG2 are initialized without safety. WDG1 is initialized with safety by calling `Wdg_17_Scu_Init` from demo.WDG1 demo is executed from CPU core 1.
- For EP devices, demo configuration WDG0 is initialized without safety.
- While configuring the GTM channel for WDGx, make sure that interrupt SRV all WDG's initialized with safety are different. Refer 3 for GTM configuration.
- The new window can be set any time using the API `Wdg_17_Scu_SetTriggerCondition()`. The maximum value of the window is 65535ms (default configuration 32000ms). Calling `Wdg_17_Scu_SetTriggerCondition()` from CPU COREx will effect the WDGx
- The mode can be changed using the API `Wdg_17_Scu_SetMode`. Calling `Wdg_17_Scu_SetMode ()` from CPU COREx will effect the WDGx
- WDG1 has to be triggered using `Wdg_17_Scu_SafeTrigger()` by passing sequence password, and `TimerCheckValue` before timeout.
- Once WDGx initialized with safety, change of mode is not allowed. If required WDGx is switched to OFF mode using `Wdg_17_Scu_SetOffMode` API and then initialized with different mode using

Wdg_17_Scu_Init() or with Wdg_17_Scu_InitAltResetPassword if alternate Password sequence password required.

9.3 Initialization of Driver

An example initialization of the WDG Driver is depicted in the [Code Listing 1](#)

Code Listing 1 WDG Driver Initialization

```

001:      /* CORE 0 EXECUTION */
002:      Void Mcal_Core0Container
003:      {
004:          /* MCU Initialization*/
005:          Mcu_Init(&(Mcu_ConfigRoot[0]));
006:
007:          /* MCU Clock / System Clock Initialization Clk: 60Mhz. */
008:          Mcu_InitClock(0);
009:          while(Mcu_GetPllStatus(0) == 0);
010:          Mcu_DistributePllClock(0);
011:
012:          /* WDG0 Initialization: WDG0 starts operating in WdgDefaultMode
            configured after call to the API */
013:          Wdg_17_Scu_Init(&Wdg_ConfigRoot[0]);
014:      }
015:
016:      /* CORE 1 EXECUTION */
017:      Void Mcal_Core1Container
018:      {
019:          #if (WDG_DELIVERY == STD_ON)
020:              Wdg_17_Scu_Core1Demo();
021:          #endif
022:      }
023:
024:      /* CORE 2 EXECUTION */
025:      Void Mcal2_Core1Container
026:      {
027:          while(ECUM_INIT_IS_NOT_COMPLETED);
028:          /* WDG2 Initialization: WDG2 starts operating in WdgDefaultMode
            configured after call to the API */
029:          Wdg_17_Scu_Init(&Wdg_ConfigRoot[0]);
030:      }

```

Code Listing 2 Example Configuration

```

031:      const Wdg_17_Scu_ConfigType Wdg_ConfigRoot[1]=
032:      {
033:
034:          {
035:              /* These are the Gtm settings for this configuration instance
            */
036:              {
037:                  {

```

```

038:                /* GTM Configuration for WDG0 */
039:
040:                15625U, /* Fast callback period in ticks. */
041:                31250U, /* Slow callback period in ticks. */
042:                50U, /* Fast callback period in ms */
043:                100U, /* Slow callback period in ms */
044:                WDG_GTM_TOM_CHANNEL, /* GTM timer module */
045:                0U, /* Gtm Module number that services H/W Wdg */
046:                2U, /* Gtm channel number that services H/W Wdg */
047:                (uint8)0U
048:
049:            },
050:            {
051:                /* GTM Configuration for WDG2 */
052:
053:                15625U, /* Fast callback period in ticks. */
054:                31250U, /* Slow callback period in ticks. */
055:                50U, /* Fast callback period in ms */
056:                100U, /* Slow callback period in ms */
057:                WDG_GTM_TOM_CHANNEL, /* GTM timer module */
058:                0U, /* Gtm Module number that services H/W Wdg */
059:                8U, /* Gtm channel number that services H/W Wdg */
060:                (uint8)0U
061:
062:            }
063:
064:        },
065:
066:
067:        /* Resolution of watchdog timer in fast mode : 0.0032 ms
068:        Minimum timeout period in fast mode : 0.0032 ms
069:        Maximum timeout period in fast mode : 209.7152 ms
070:        */
071:        {
072:
073:            (uint16)3036U, /* Fast Mode Reload Value for WDG0 */
074:            (uint16)3036U /* Fast Mode Reload Value for WDG2 */
075:
076:        },
077:
078:
079:        /* Resolution of watchdog timer in slow mode : 0.2048 ms
080:        Minimum timeout period in slow mode : 0.2048 ms
081:        Maximum timeout period in slow mode : 13421.7728 ms
082:        */
083:        {
084:            (uint16)46004U, /* Slow Mode Reload Value for WDG0 */
085:            (uint16)46004U /* Slow Mode Reload Value for WDG2 */
086:

```

```

087:         },
088:
089:
090:
091:         {
092:             (uint16)46004U, /* Slow Mode Reload Value for WDG1 */
093:         },
094:
095:
096:
097:         /* Initial password */
098:
099:         {
100:             (0x0078U), /*Initial password of WDG1 */
101:         },
102:         /* Configuration : WdgDefaultMode:
103:             Default mode for watchdog driver initialization: WDG083:
104:             - WDGIF_OFF_MODE
105:             - WDGIF_SLOW_MODE
106:             - WDGIF_FAST_MODE
107:         */
108:
109:
110:
111:         {
112:             WDGIF_OFF_MODE, /*Default Mode for WDG0 */
113:             WDGIF_SLOW_MODE, /*Default Mode for WDG1 */
114:             WDGIF_OFF_MODE /*Default Mode for WDG2 */
115:         },
116:
117:         /* TimerTolerance */
118:         {
119:             (0x03U), /*Timer tolerance of WDG1 */
120:         },
121:         /*WdgSafetyProperty , 1 = WDGx safe; 0 WDGx QM */
122:         {
123:             (0U),
124:             (1U),
125:             (0U),
126:         },
127:         /* WdgSafeIndex */
128:         {
129:             (255U),
130:             (0U),
131:             (255U),
132:         },
133:         /* WdgQmIndex*/
134:         {
135:             (0U),
136:             (255U),

```



```

0137:          (1U),
0138:      }
0139:      },
0140:
0141:
0142:      };

```

To calculate new 14-bit LFSR (Linear Feedback Shift Register) with characteristic polynomial $x^{14}+x^{13}+x^{12}+x^2+1$.

Code Listing 3 shows sequence password calculation in which Pass_next is a 14-bit value(WDTxCON0.PW) and it holds current password which is used to calculate next sequence password(Expected_Pass) to unlock WDTCON0. The Expected_Pass is stored in Pass_next to be used to calculate next sequence password.

Code Listing 3 Password calculation

```

0143:      {
0144:          Password = ((Pass_next >> 13U) ^
0145:                      (Pass_next >> 12U) ^
0146:                      (Pass_next >> 11U) ^
0147:                      (Pass_next >> 1U)) & 1U;
0148:          Expected_Pass = ((Pass_next << 1U) |
0149:                          (Password)) & 0x00003FFFU;
0150:          Pass_next = Expected_Pass;
0151:      }

```

9.4 Using the API

9.4.1 Changing Mode of Operation

The **Code Listing 4** shows an example of changing mode of operation for WDGx enabled with safety.

Code Listing 4 Change mode operation

```

0152:      /* Change mode of operation to WDGIF_FAST_MODE*/
0153:      Wdg_17_Scu_SetMode(WDGIF_FAST_MODE);

```

9.4.2 Triggering Watchdog

The Code Listing below shows the way of triggering the WDG enabled without safety.

Code Listing 5 Triggering WDG

```

0154:      Wdg_17_Scu_SetMode(WDGIF_SLOW_MODE);
0155:      Wdg_17_Scu_SetTriggerCondition(5000);

```

The above code sets the WDG in SLOW MODE. The WDG timeout window is configured for 5000ms.

The WDG will expire after 5000ms and trigger the SMU.

Code Listing 6 shows the way of triggering the WDG enabled with safety. SeqPass and TimerCheckValue are passed to unlock WDTCON0 register while password access. ReqPass to be sent while modify access. This functionality triggers WDG in current mode. In given demo example WDGx 14 bit password field is initialized with 0x0078. This value is shifted twice and written into password field. The next sequence of passwords are 0x00F0U, 0x01E0U, 0x03C0U, 0x0780U, 0x0F00U, 0x1E01U etc.

Code Listing 6 Triggering WDG with safety

```
0156:                Wdg_17_Scu_SafeTrigger(SeqPass, TimerCheckvalue);
```

Code Listing 6 triggers WDGx with safety

9.4.3 Disabling Watchdog

The **Code Listing 7** shows an example to disable WDG. This operation is possible only if WdgDisableAllowed is TRUE.

Code Listing 7 Disable Watchdog

```
001:                /* Disable Watchdog */  
002:                Wdg_17_Scu_SetMode(WDGIF_OFF_MODE);
```

Note: In Slow/Fast mode of operation WDG needs to be triggered within the window value using the API Wdg_17_Scu_SetTriggerContidition () to avoid triggering of SMU from Watchdog overflow.

9.4.4 Set Watchdog to OFF mode

Code Listing 8 shows an example to disable WDG and disable WDG safety features. This functionality is used if WDGx enabled with safety has to be disabled. SeqPass1 is calculated from previous password which was used for previous password access. SeqPass2 is calculated from SeqPass1.

Code Listing 8 SetOffMode Watchdog

```
003:                /* SetOffMode Watchdog */  
004:                Wdg_17_SetOffMode(SeqPass1, SeqPass2);
```

10 Safety Information

WDG driver supports program flow monitoring by providing following safety features

- Automatic password sequencing – code sequence checking(logical supervision)
- Execution time checking – This mechanism can provide a check of the elapsed program execution time since the last WDT restart (temporal supervision). This functionality is optional.

A container WdgSafety is added to enable each of the WDGx for safety usage.

10.1 Safety Mechanisms

- Regular AUTOSAR WDG is considered as complete QM, and WDG with safety is possible only with password access and execution time check.
- It is possible to configure one of the WDG0 without safety and WDG1 with safety. Wdg_17_Scu_Init() is used to initialize WDGx with or without safety.
- Wdg_17_Scu_Init() function enable safety features (password sequencing and execution time checking) if the WDGx is safety enabled in 4.2.11. The initial password and tolerance time is provided in configuration.DEM is reported for wrong WDG safe initialization. Refer Figure 6 for Wdg initialization.
- The WDG driver provides information to the integrator about the SFR memory regions, data regions and code region used by driver. This information helps the integrator to define the proper MPU settings for WDG access. WDTCON1 register and its configuration value is provided to integrator.
- Wdg_17_Scu_SafeTrigger() is used to trigger WDG with safety enabled. This functionality requires sequence password and estimated service time as input parameters. Execution time checking is made optional. WDG servicing include password access followed by modify access. During Password access, if WdgEnableTimecheck is STD_ON then reload value is written with estimated service time within tolerance else current reload value is written back to unlock WDTCON0. Sequence password is used to unlock WDTCON0 register. During modify access, reload value is updated. To unlock the WDTCON0 register next time, sequence password is calculated from the new password. Wrong password access / wrong servicing time sets an SMU alarm by WDG hardware
- Wdg_17_Scu_SetOffMode is used to switch WDG to OFF mode and disable safety features. This functionality requires 2 consecutive passwords. Ideally even the estimated service time has to be given from the application layer, however considering the complexity involved and the use case to call this API is rare, execution time checking is performed by reading the timer and use the same rather than having it as a parameter
- The API Wdg_17_Scu_InitAltResetPassword is provided to user, for cases where user wishes to start a new program flow check with an alternate initial password while software is executing. In such scenario user has to switch off the watchdog WDGx with Wdg_17_Scu_SetOffMode API, and then use Wdg_17_Scu_InitAltResetPassword API to initialize watchdog with new initial password (reset password).

10.2 Condition of use

- Wdg_17_Scu_Init() shall be the last possible low level driver module initialization (that access endinit protected registers) from the application to avoid further calls of Mcal_ResetENDINIT() / Mcal_SetENDINIT().
- WDG shall be initialized either in SLOW/FAST mode and shall not be initialized in OFF mode.
- User shall use SafeTLib software to periodically check WDTCON0 register configuration

- User shall calculate sequence password prior to Wdg_17_Scu_SafeTrigger(). Current password is used to calculate sequence password to unlock WDTCON0.
- User shall set the MPU region for the used SFR's and variable region for protecting against corruption of variable and to ensure only the appropriate caller is used
- Wdg_17_Scu_SetOffMode() is provided only to assist the application in special corner cases and it is expected to be used judiciously

Scenario 1

Modes: SLOW -> OFF, FAST -> OFF

User shall calculate next two consecutive passwords prior to Wdg_17_Scu_SetOffMode().

Password1 is calculated from current password and Password2 is calculated from Password1.

Scenario 2 –

Modes: OFF -> SLOW, OFF -> FAST

Wdg_17_Scu_Init() function shall be called to enable safety features and to set mode. Later to unlock WDTCON0 password shall be calculated from the initial password

Scenario 3

User intends to reset password sequence, then user shall call Wdg_17_Scu_SetOffMode() followed by Wdg_17_Scu_InitAltResetPassword(), with desired reset password in second argument of the API.

- During run time if user needs to access Mcal_ResetENDINIT() / Mcal_SetENDINIT(), WDG shall be disabled. **Figure 7** explains access to Mcal_ResetENDINIT()/Mcal_SetENDINIT() after WDG initialization.

10.3 Sequence diagrams

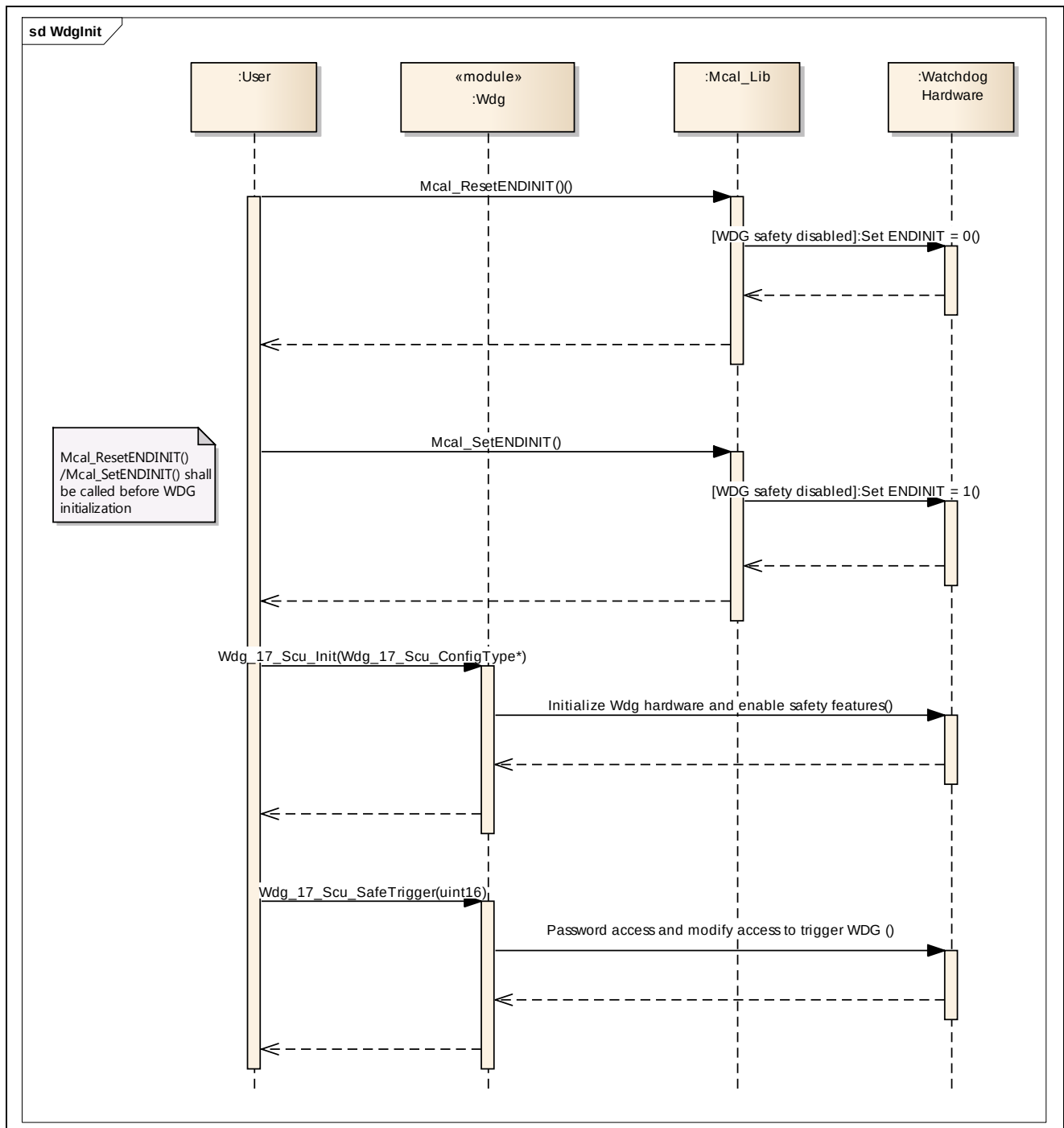


Figure 6 WDG initialization

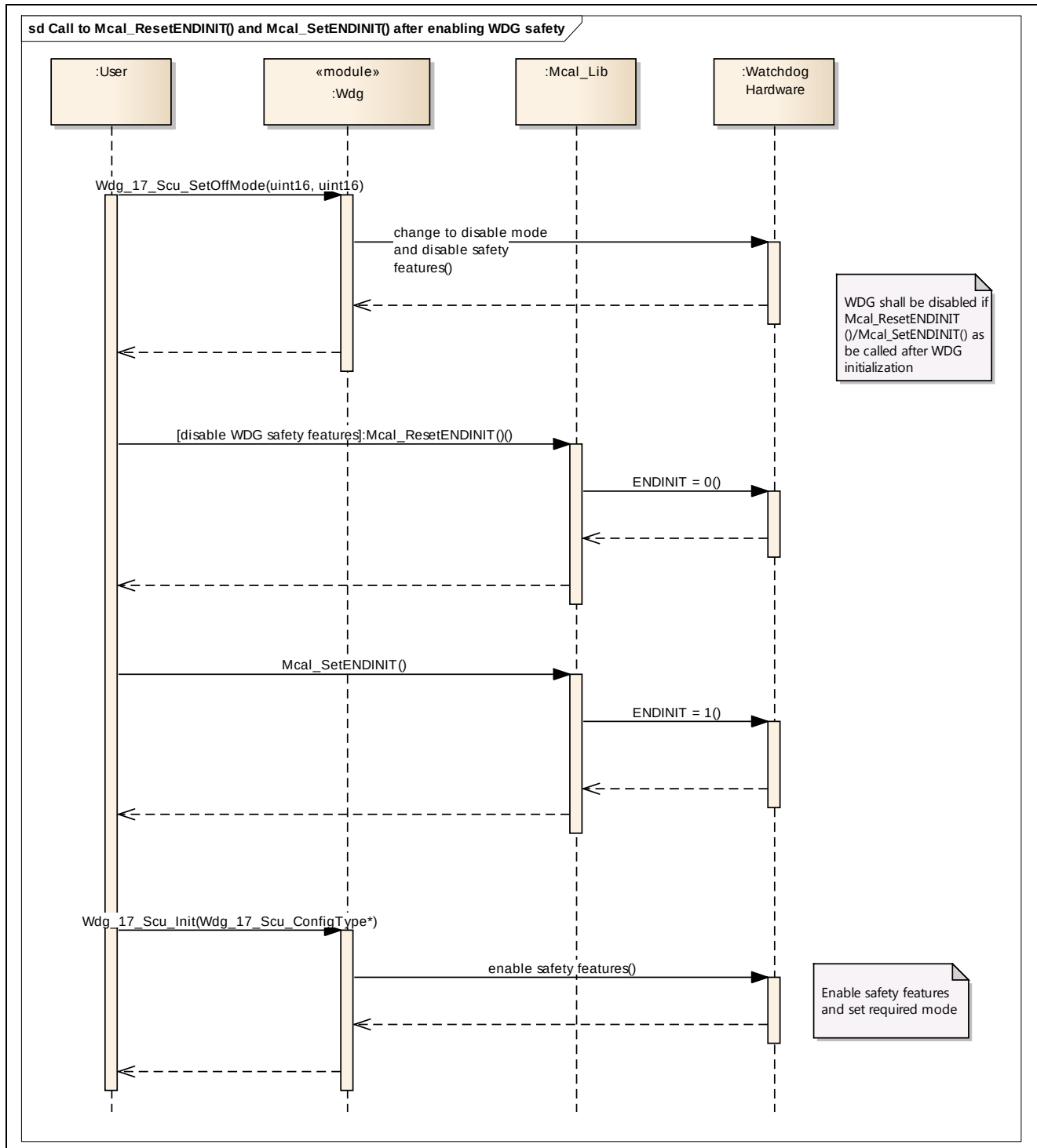


Figure 7 Access to Mcal_ResetENDINIT()/Mcal_SetENDINIT() after WDG initialization

11 Assumptions and Limitations

11.1 Assumptions and Deviations from the Software Specification

The WDG driver does not support following features mentioned in the specification [1](#)

- WDG093 and WDG094 (Section – 8.3.3) requirement from specification is not supported as AURIX microcontrollers does not have this hardware feature.
- WDG074_Conf (Section – 10.3) for WDG_TRIGGER_MODE requirement from specification is not supported as AURIX microcontrollers does not have toggle/window mode of operation.
- It is assumed that the user is aware of the dangers involved in changing the clock frequency during runtime as Wdg module is highly clock timing dependent (Viz. H/W WDT and Gtm)
- Though WDG168 provides an option to execute the watchdog driver from RAM, this feature is not implemented in this release. WDG module always executed from ROM.
- Std_Types.h is included directly into Wdg_17_Scu.h

11.2 Limitations

If WDG with safety is enabled then:

1. The complete windowed watchdog concept will not be supported with respect to AUTOSAR 4.0.3.
2. Execution time checking is made optional.
3. WDG with safety will not be initialized in OFF mode.
4. All calls to Mcal_ResetENDINIT()/Mcal_SetENDINIT() shall be avoided after WDG initialization. If necessary Wdg_SetOffMode() shall be called and then call Wdg_17_Scu_Init().
5. DET check is available for Wdg_17_Scu_Init() function only.
6. For EP devices, since there is only one Watchdog Timer, either full AUTOSAR functionality or safety implementation can be selected. As a result, when safety is selected AUTOSAR API other than Wdg_17_Scu_Init will not be visible. Similarly safety APIs are not available if safety feature is disabled.

11.3 Hardware Features Not Supported

This section lists down all the hardware features that are not supported by the WDG driver. These features are out of the scope of WDG driver's specification [1](#).

- Check access is not supported.
- Mode to set input frequency for WDT's to $f_{SPB}/64$ is not supported.

www.infineon.com